

MeritScreen

Parental-control launcher for children aged 3–12 — product overview, onboarding & authentication, data model & flows, screens, architecture, and AI adaptive quiz with concept explanations.

Platform focus: Android Kotlin native launcher · Firebase backend · iOS controlled environment

Documents included: 01–06 product & data flow set

Generated from: /docs markdown sources

Contents

- 1. 1. 01 — Product overview
- 2. 2. 02 — Onboarding & authentication
- 3. 3. 03 — Data model & data flow
- 4. 4. 04 — Screens & features
- 5. 5. 05 — Architecture & performance
- 6. 6. 06 — AI adaptive difficulty and concept explanations
- 7. 7. 07 — App blocks, quiz gate, and fail lock

Diagrams are Mermaid flow, sequence, and entity-relationship charts rendered from the source documents. Tables list screens, data fields, and architecture choices.

1.01 — Product overview

What MeritScreen is

MeritScreen is a **parental-control launcher** for children aged **3–12**. The same rules run on Android and iOS. The shell is different because the platforms are different.

- **Android:** MeritScreen **is the default Home app**. Home or unlock opens our launcher, not Samsung One UI Home, Pixel Launcher, or Oppo Launcher. The child does not get the stock app drawer or a widget wall. They see approved apps, remaining time, and quizzes.
- **iOS:** Apple does not allow a custom Home. The app is a **controlled environment** built in **SwiftUI**, using Screen Time / Family Controls (`FamilyControls` , `ManagedSettings` , `DeviceActivity`). Blocked apps are shielded. The quiz and parent dashboard are SwiftUI.

The parent manages everything from **their own phone**. The child never creates an account.

Alignment with the product brief

Brief requirement	Spec
Ages 3–12, Android + iOS	Yes. Age bands 3–6, 7–9, 10–12
Android replaces stock launcher	Yes. <code>HOME</code> / <code>ROLE_HOME</code> . No stock drawer
iOS uses Screen Time / Family Controls	Yes. SwiftUI + Family Controls. Not a fake launcher
Parent: allow apps, limits, quiz mode, rewards, reports	Yes. Remote dashboard
Child: approved apps, time left, quiz	Yes
Pass → more time on that app + rewards	Yes. Another block of that app (example: +30 min YouTube)
Fail → cooldown set by parent; apps shielded until cooldown or retry	Yes. All non-emergency apps shield immediately, including the one in use. Only Phone stays open. Unlock = cooldown ends or a retry is passed. See 07
Adaptive quiz + concept explanation on a wrong answer	Yes. 06
Kotlin launcher, SwiftUI, Firebase	Yes. 05
COPPA / Play Families / App Store kids	Yes
Out of v1: GPS, web filter, child social	Yes

The promise

Goal	How the app delivers it
Safe	Only parent-approved apps. Stock drawer is not available on Android
Limited	Each app has a time block . Optional daily ceiling
Fair fail	Failed quiz stops all further usage. Only emergency apps stay open until cooldown or a passed retry
Educational	Adaptive quiz. Wrong tap explains the choice and the concept
Remote	Parent changes rules without touching the child's phone
Light	Child UI is native, local-first, offline capable

Who uses it

Person	Device	How they enter
Parent	Parent's phone (Android or iOS)	Email / Google / Apple (Firebase Auth)
Child	Child's phone	No password. Parent pairs with QR or 6-digit code

Age bands (quiz difficulty)

Age band	Quiz style
3–6	Colors, shapes, matching, simple counting
7–9	Basic math, spelling, reading
10–12	General knowledge, reasoning, slightly harder math

The parent picks the band. Difficulty then **moves inside that band** from the child's answers. A wrong answer teaches the concept. The engine does not promote the child into another age band by itself.

What the parent can control

1. **Approve apps** the child may open.
2. Set a **time block per app** (example: YouTube 30 minutes, then a quiz).
3. Set **cooldown on fail** (example: 15 minutes). A fail locks **all** non-emergency apps. A passed **retry** ends the lock early.
4. Optional **daily ceiling** across all apps.
5. **Quiz trigger mode**: app block (recommended), every session, or daily ceiling.
6. **Rewards**: extra minutes (the next block), stickers, weekend bonus.
7. **Reports**: usage by app, quiz results, weak topics, which apps are in cooldown.

What the child sees

- Approved apps only (Android Home grid; iOS allowed set + shields).
- Time left on the app they are using.
- A quiz when the block (or other rule) says so.
- **Pass** → that app continues for another block, plus reward if configured.
- **Fail** → the launcher shields **all apps**, including the one they were using. Only emergency apps such as Phone stay open until the cooldown ends or they pass a retry quiz.
- Wrong answers: why that choice is wrong, then the concept, in short age-fit language.

Worked example and shield rules: [07 — App blocks and fail lock](#).

Why the stacks differ

Android is Kotlin + Jetpack Compose because the process that answers Home must start fast, stay small, and work offline. AI does not run on that path.

iOS is **SwiftUI** (Swift) for every screen: parent dashboard, child quiz, shields, and pairing. Family Controls does the shielding the launcher does on Android. Same Firebase project and the same policy JSON so a family can mix an Android child phone and an iOS parent phone.

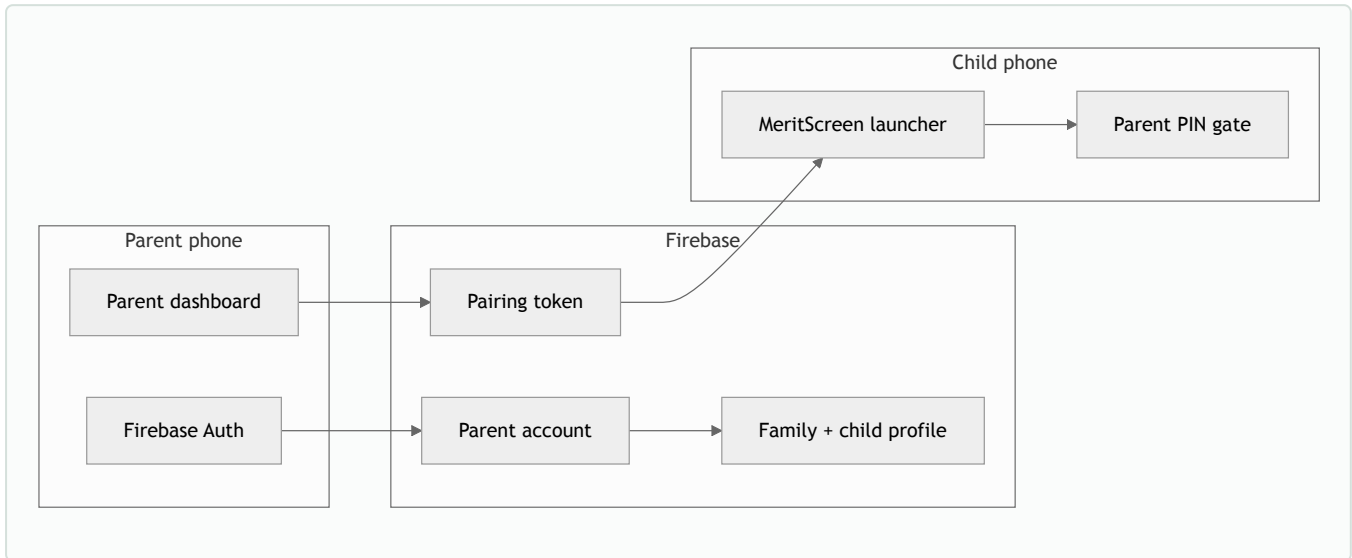
Out of scope for v1

- Live GPS tracking
- Full web filtering / DNS VPN
- Child social features or chat
- Behavioral ads
- Storing the child's photos or contacts

2. 02 — Onboarding & authentication

This document explains **how a family starts using MeritScreen** and **who is allowed to do what**.

There is one authenticated adult. The child's phone is a **paired device**, not a second login.



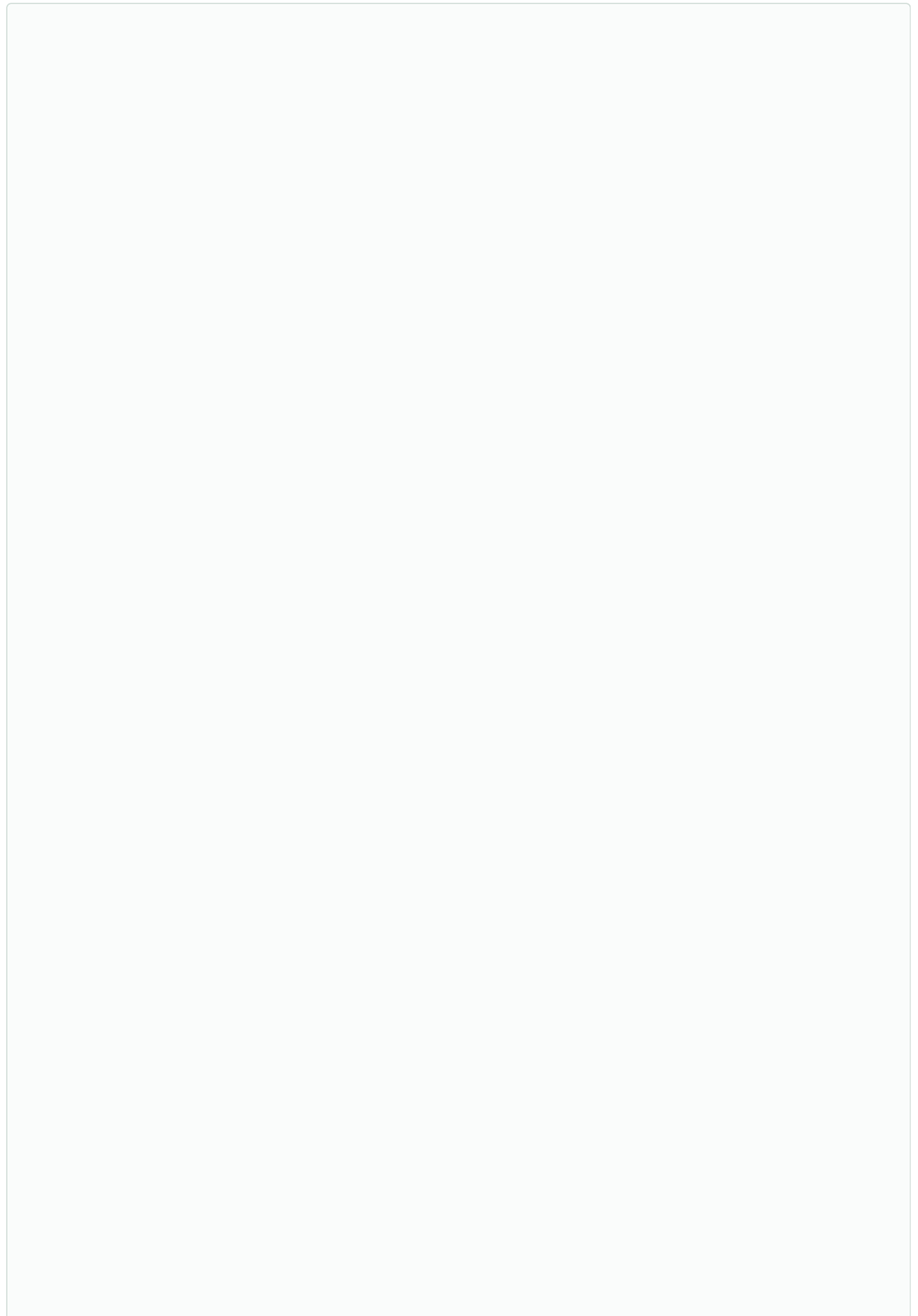
Flow diagram 1

Roles and access

Role	Authenticates with	Can do
Parent	Email + password, Google	Create family, add children, pair devices, change rules, view reports, set PIN
Child	Nothing. Device holds a pairing token	Open approved apps, take quizzes, see remaining time
Parent-on-child-device	4–6 digit Parent PIN	Exit launcher, open settings, unpair, change allowed apps on-device

The Parent PIN is stored as a **hash** (never plain text). It is synced to the child device so a parent can recover control even when offline.

Parent onboarding (first 5 minutes)



**Install MeritScreen on
parent phone**



Choose I am a parent

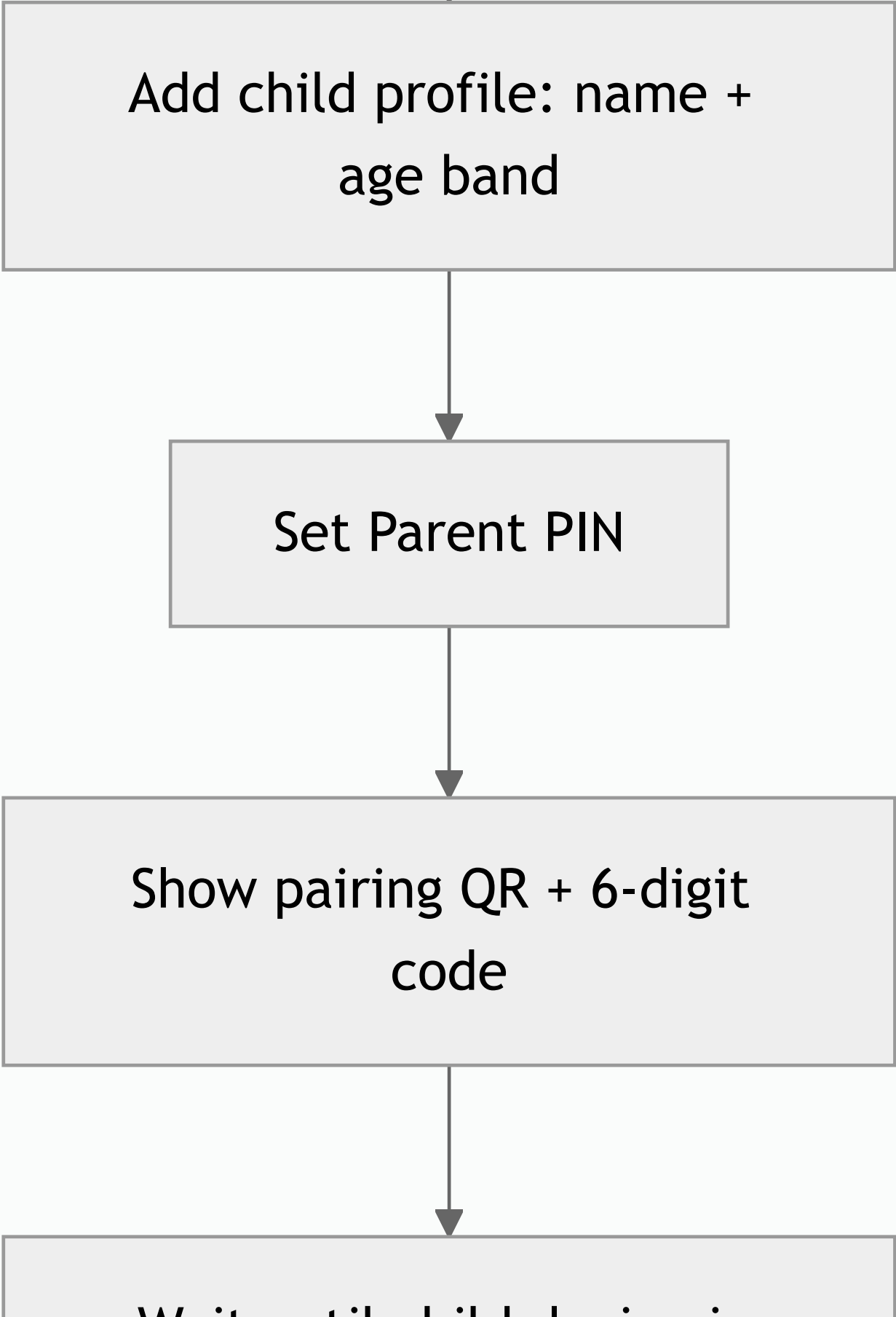


Sign up / Sign in



Create family





```
graph TD; A[ ] --> B[Add child profile: name + age band]; B --> C[Set Parent PIN]; C --> D[Show pairing QR + 6-digit code]; D --> E[Wait until child device is...];
```

Add child profile: name +
age band

Set Parent PIN

Show pairing QR + 6-digit
code

Wait until child device is...

Wait until child device is

Flow diagram 2

paired

Step by step

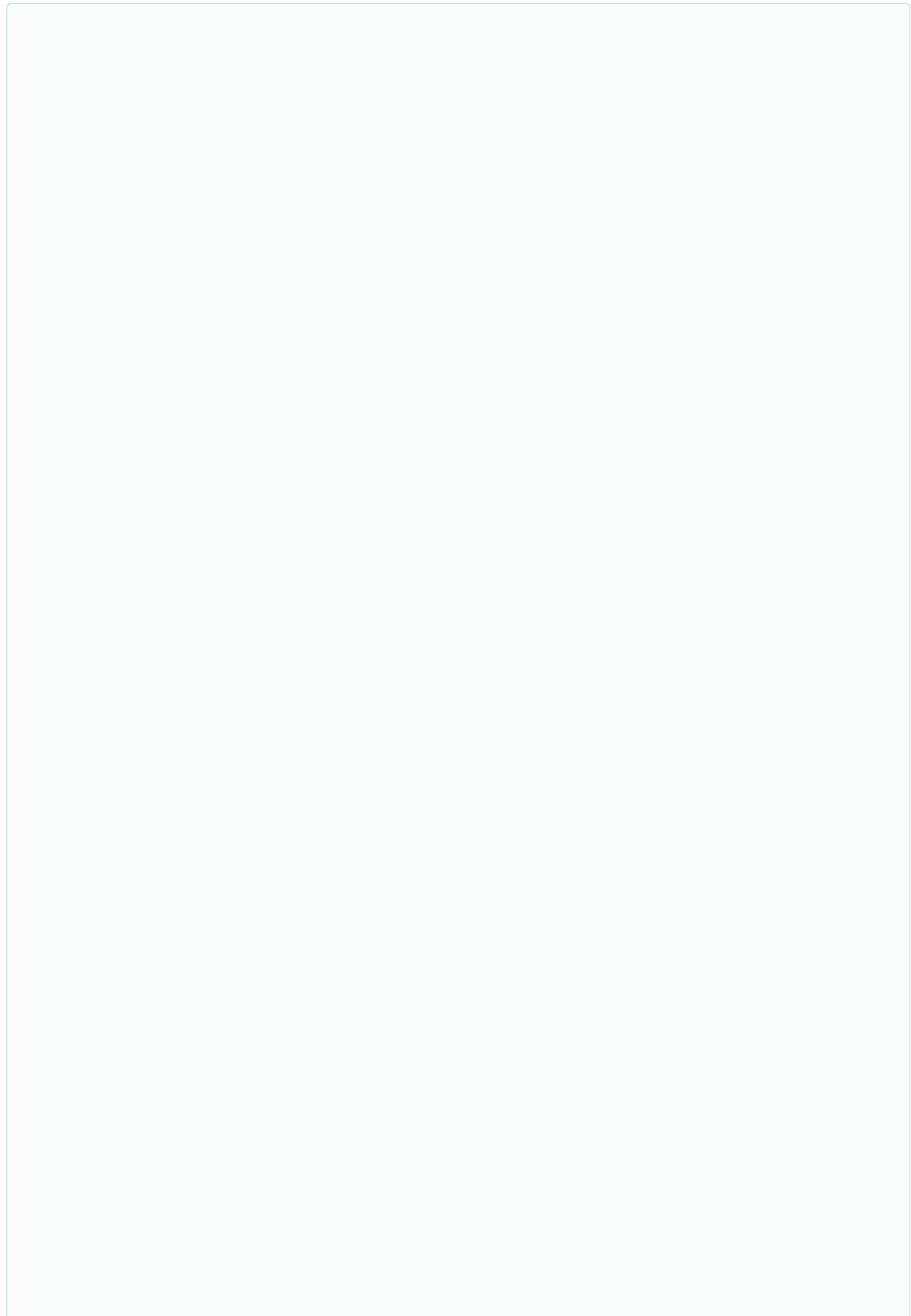
1. **Install** the parent app (same binary can detect role, or a clear first screen: *Parent / Set up child device*).
2. **Sign up** with email, Google, or Apple. Firebase creates `uid`.
3. **Create a family**. One parent is the owner. (Invite a second parent can wait for v1.1.)
4. **Add a child profile**: display name, age band (3–6 / 7–9 / 10–12), language, avatar from a preset pack (no camera required).
5. **Set Parent PIN**. This PIN unlocks the child's launcher settings.
6. **Pairing screen** shows:
 - QR code containing a short-lived pairing token
 - 6-digit code as a fallback
 - Token expires in ~10 minutes; parent can refresh
7. After the child device confirms, parent **picks allowed apps, daily minutes, and quiz mode**.
8. Dashboard is ready. Push notifications are requested here (usage alerts, quiz summaries).

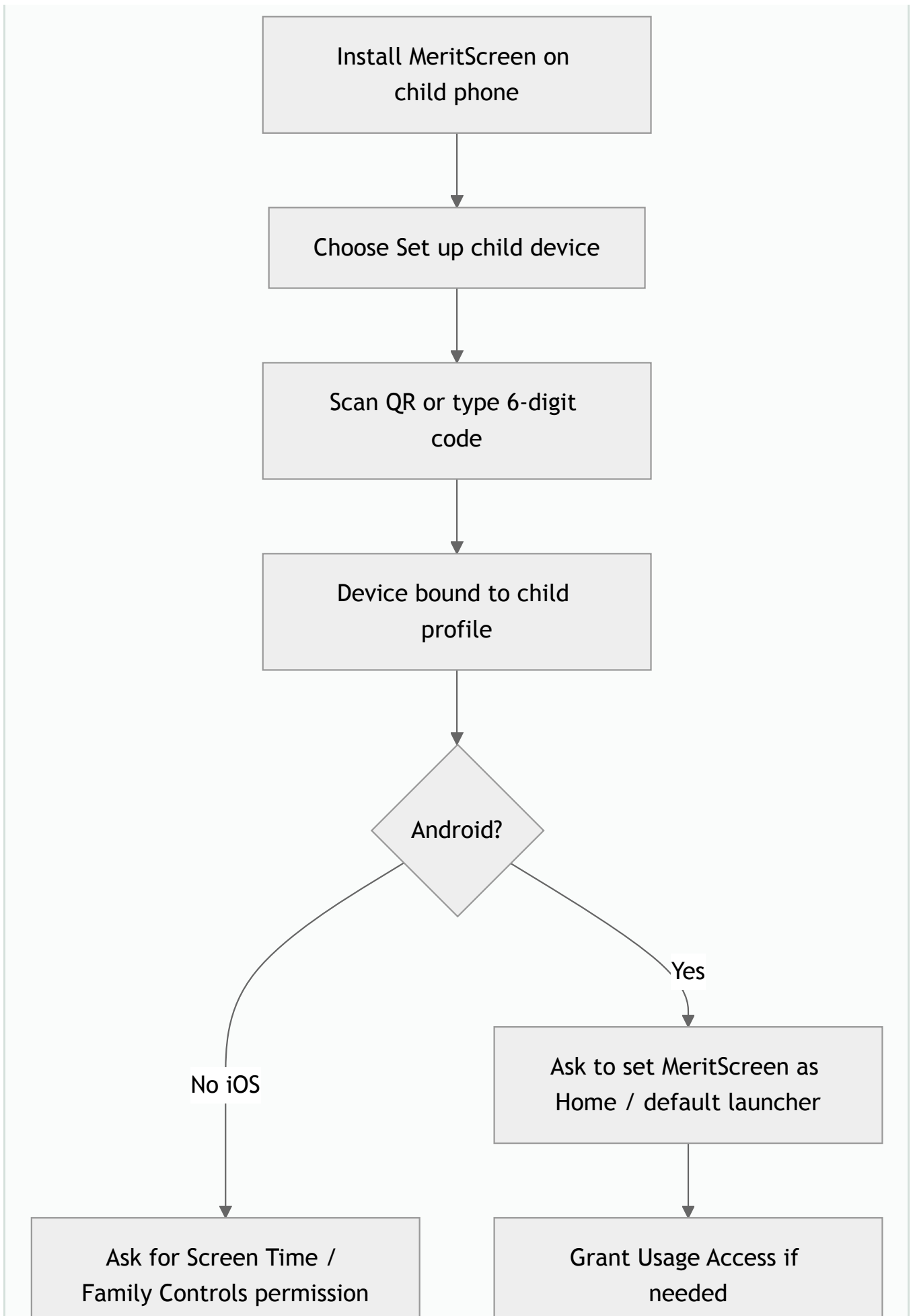
Set allowed apps, daily limit, quiz mode

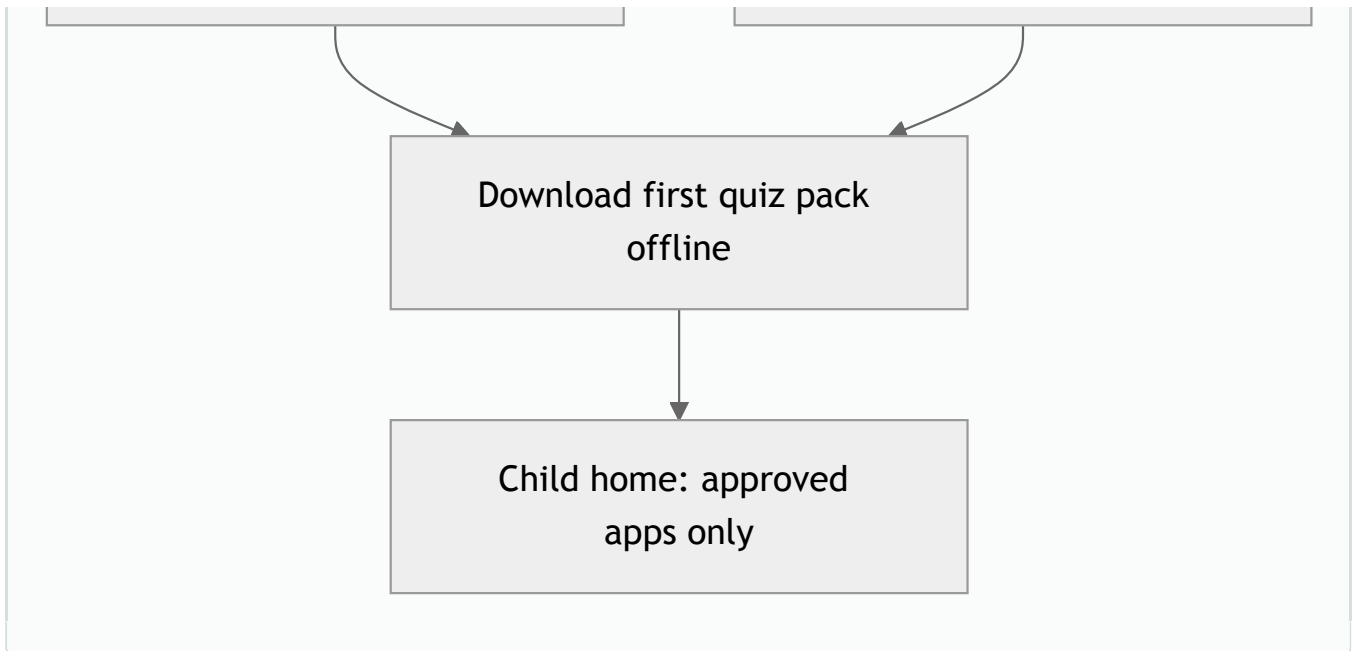
Child device onboarding

The parent physically has the child's phone for this part.

Parent dashboard home







Flow diagram 3

Android launcher setup (required)

1. System prompt: **Set MeritScreen as Home app**.
2. Parent confirms. From now on, Home / unlock opens MeritScreen.
3. Optional but recommended: Usage Access so we can report time even if an app was opened from elsewhere.
4. Parent PIN is required to:
 - switch back to the stock launcher
 - uninstall MeritScreen (we cannot fully block uninstall without Device Owner / MDM; we **discourage** it and detect if the app is no longer default Home)
5. First quiz pack (~20–50 questions for that age band) downloads and is stored in local Room DB.

iOS setup (controlled environment)

1. Parent authorizes **Family Controls / Screen Time** for the child Apple ID or this device.
2. MeritScreen applies Managed Settings (allowed apps, shield).
3. Same pairing and same cloud family. No custom home screen.

Authentication design

Parent (cloud)

Item	Choice
Provider	Firebase Authentication
Methods	Email/password, Google, Sign in with Apple
Session	Firebase ID token, refreshed automatically
Multi-device	Yes — parent can open the dashboard on another phone after sign-in
Sign-out	Clears local cache of reports; does not unpair child devices

Child (device pairing)

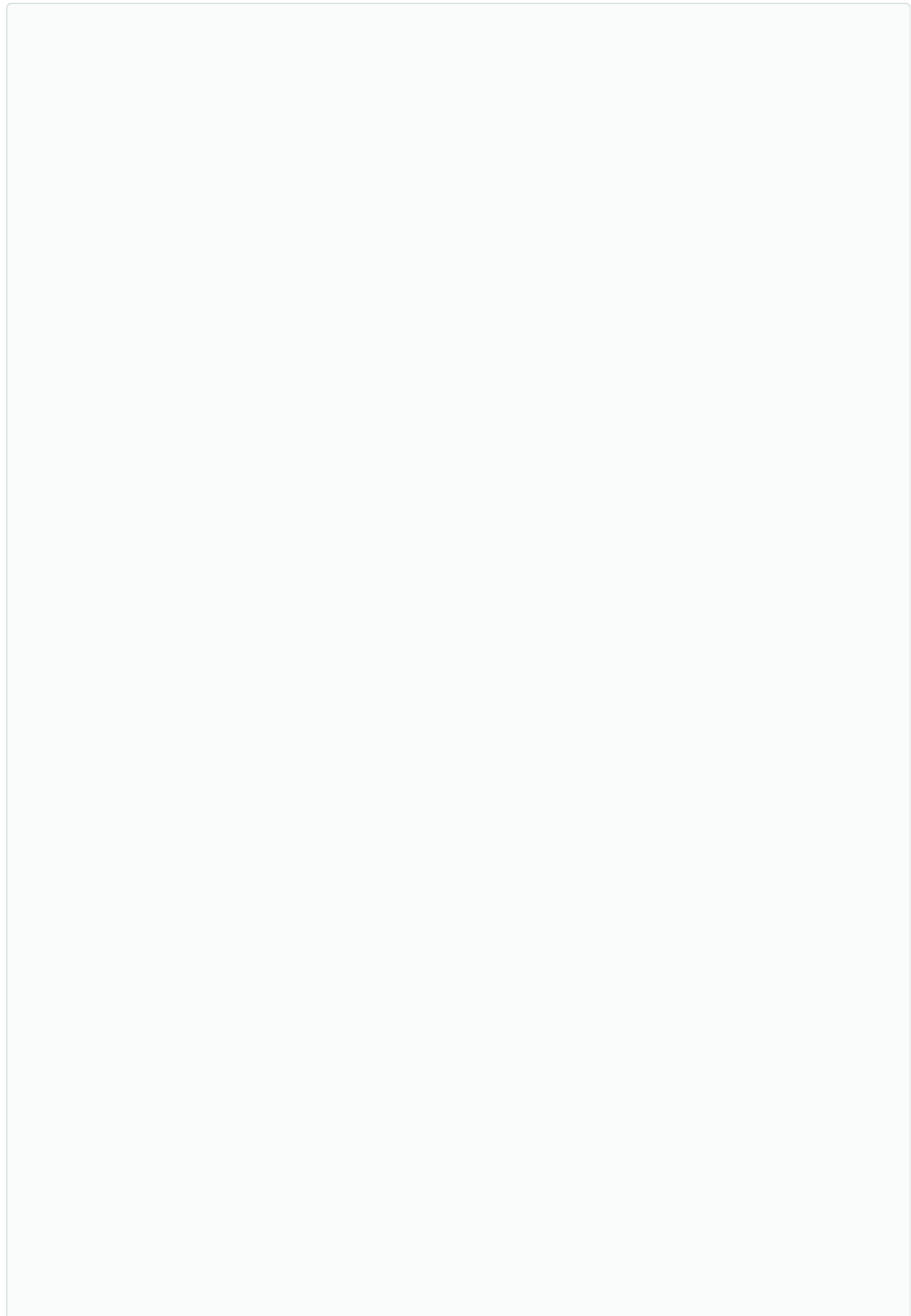
Item	Choice
No email / no password for the child	Required for kids privacy
Pairing token	Created in Cloud Functions, stored hashed, one-time use
Device record	<code>deviceId</code> + <code>childId</code> + platform in Firestore
Local secret	Encrypted pairing credential in Android Keystore / iOS Keychain
Re-pair	Parent can revoke a device from the dashboard; child must scan again

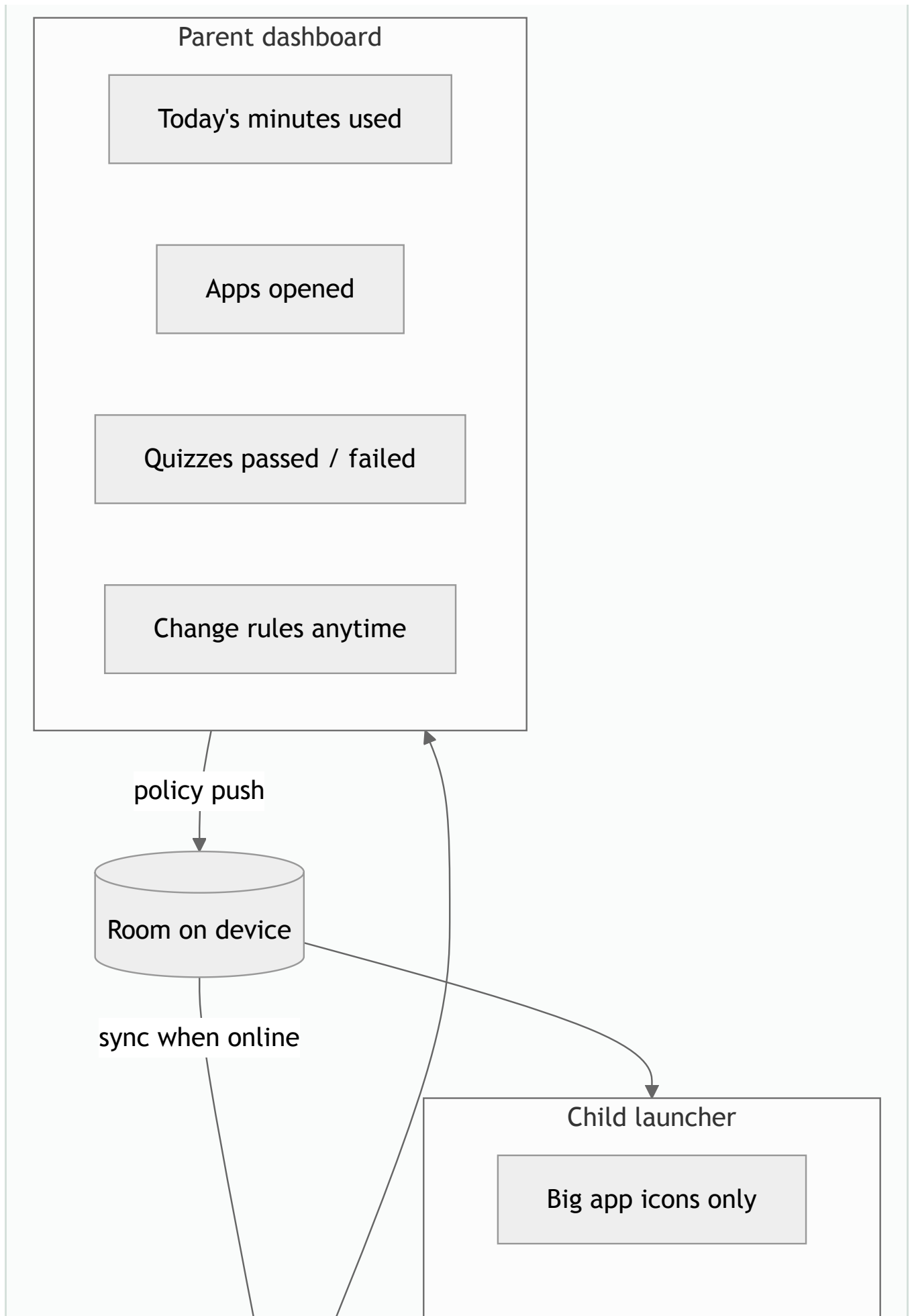
Parent PIN (on-device gate)

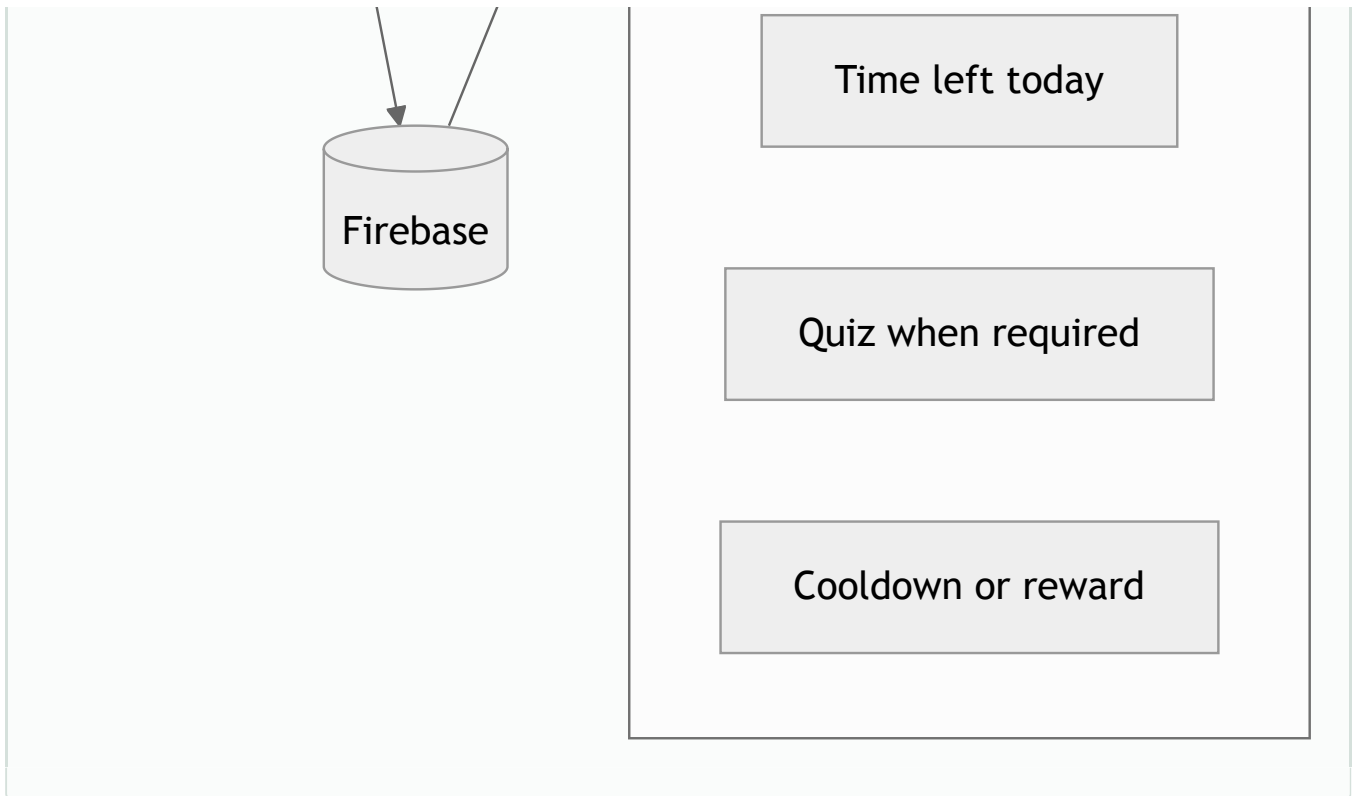
Used **only on the child device** for adult override.

- 4–6 digits, chosen by parent
- PBKDF2 / Android Keystore-backed verification
- After 5 failures: 5-minute lock (shown in a calm, non-scary way)
- Forgot PIN: parent signs in on **their** phone and resets; the new hash syncs to the child device

After onboarding: who sees what







Flow diagram 4

- **Parent** sees summaries, not a live video of the screen.
- **Child** sees a calm home. They do not see the parent's email, other children, or billing.
- **Rules** flow parent → cloud → child. **Usage and quiz scores** flow child → cloud → parent.

Failure cases (keep onboarding unstuck)

Problem	What the app does
QR expired	Parent taps Refresh; new code in 1 second
No internet on child during pair	Pairing needs one short online check; then the launcher works offline
Parent skips "set as default launcher"	Child home still works inside the app, but stock Home is reachable — show a persistent "Finish setup" card
Wrong age band	Parent can change it later; next quiz pack regenerates
Second child	Repeat "Add child" + pair a second device; same parent account

3. 03 — Data model & data flow

This document answers three questions:

1. What information do we **store**?
2. How does that information **move** between parent, child, and cloud?
3. What do we **never** collect?

The child launcher is **offline-first**. The phone must stay useful in airplane mode. Cloud sync is a background job, not a requirement to open Home.

Data we collect

Parent information

Field	Why	Required
Email / Google / Apple id	Sign-in	Yes
Display name	Dashboard greeting	Optional
Firebase uid	Account key	Yes (generated)
FCM push token	Alerts	Optional until permission
Parent PIN hash	Unlock child-device settings	Yes
Notification preferences	Quiet hours, daily summary	Optional

We do **not** need the parent's contacts, photos, or location.

Child information (profile — not an account)

Field	Why	Required
Display name (can be a nickname)	Show on launcher and reports	Yes
Age band (3–6, 7–9, 10–12)	Quiz difficulty	Yes
Birth year (optional, not full birthday)	Fine-tune age band	No
Language	Quiz language	Yes (default device language)
Preset avatar id	Friendly home screen	Yes
Device id + platform	Pairing	Yes after setup

We do **not** collect: school name, real-time location, contacts, photos, microphone, or a child email.

Productivity & usage data (from the child device)

Event	What it contains	Used for
Session start / end	Time, device	Daily minutes
App opened	Package name / bundle id, duration	“Which apps today”
Minutes remaining	Computed from policy + usage	Child overlay + parent dashboard
Quiz attempt	Topic, score, pass/fail, extra minutes granted, level before/after	Learning + adaptive difficulty
Launcher still default?	Boolean (Android)	Setup health
Policy version applied	Number	Confirm child device received new rules

Usage is **aggregated per app per day**, not a keystroke log and not a screenshot stream.

Quiz / AI data

Field	Purpose
Question id, prompt, choices, correct answer	Show quiz offline
<code>whyCorrect</code> , <code>whyWrongByChoice</code> , <code>conceptExplainer</code>	Teach immediately after each tap, still offline
Source: <code>bank</code> or <code>ai</code>	Quality control
Topic + <code>conceptId</code> + difficulty 1–5	Pick the next item near current skill
Child’s answer + time-to-answer	Score, pacing, skill update
<code>skill_state</code> per topic (level, streaks, weak concepts)	Next question this session + next AI pack
Weak topics / concepts	Next pack practices what they missed

AI prompts never include the child’s name, email, or raw usage history.

The cloud job receives only: age band, language, skill snapshot (levels + weak concept ids), and “avoid these question ids”.

Full engine: [06 — Adaptive quiz & explanations](#).

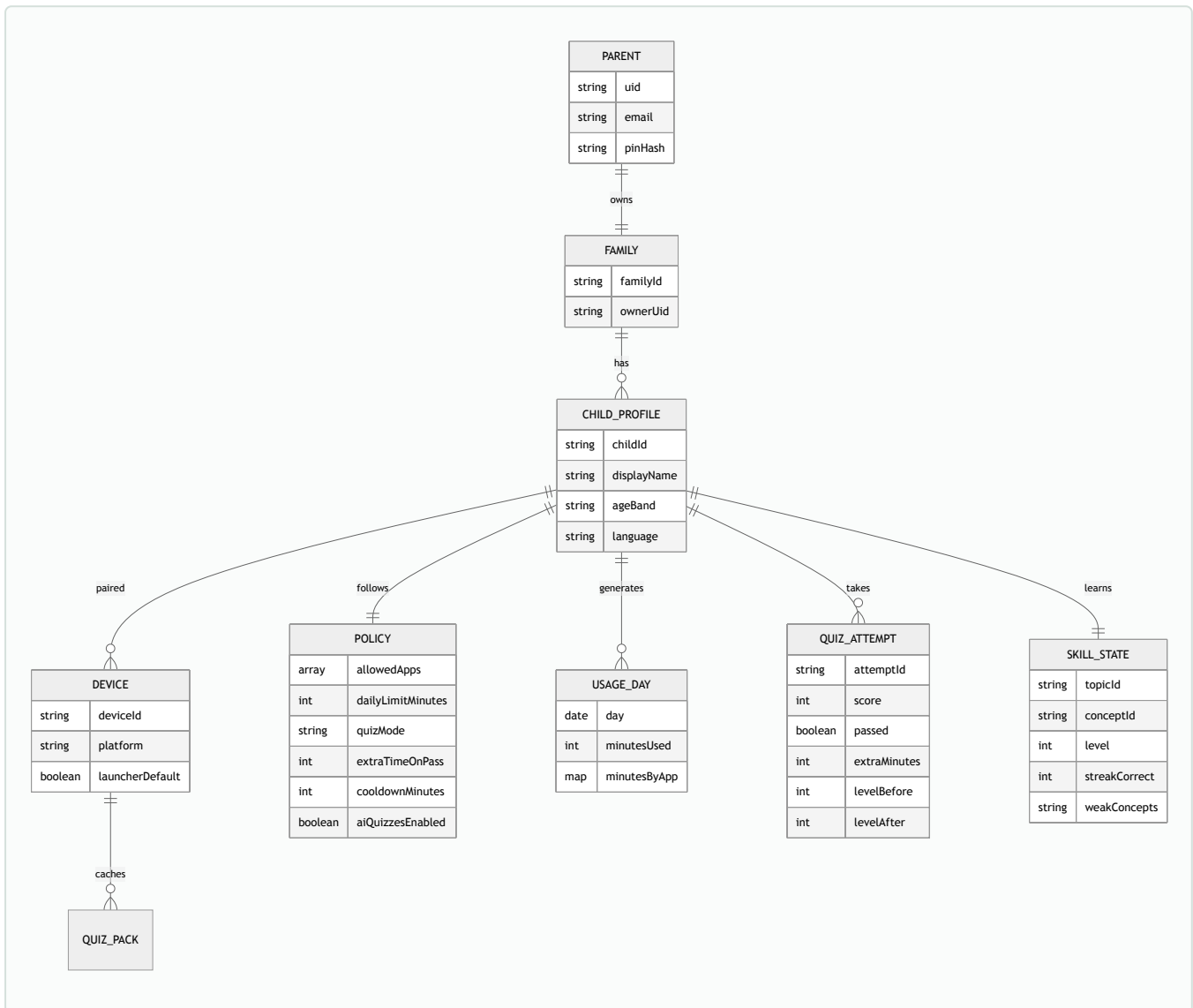
What we do not collect

- Precise GPS / location history
- SMS, call logs, contacts
- Photos or camera roll
- Full browsing history inside third-party apps
- Advertising IDs for ads (no ads in the child launcher)

- Voice recordings
- Face photos for “child recognition”

This is both a privacy choice and a Play / App Store kids-policy requirement.

Core entities



Flow diagram 1

Policy (the rules object)

One document per child, plus one row per allowed app. The parent dashboard writes them. The child device applies them locally. Full enforcement: [07 — App blocks and fail lock](#).

Child-level

Setting	Example	Effect on child
quizMode	app_block (also every_session , daily_ceiling)	When a quiz appears
failLockScope	all_non_emergency	A fail shields every approved app immediately
allowRetryDuringCooldown	true	A passed retry ends the lock early. Required unlock path
dailyCeilingMinutes	off, or 120	Stops further blocks when today's total is used
questionsPerQuiz	3–5	Keep it short for ages 3–6
passScorePercent	70	Pass / fail
rewardsEnabled	true	Sticker / weekend bonus
aiQuizzesEnabled	true	Cloud fills packs; bank is fallback
adaptiveDifficultyEnabled	true	Rungs move after each answer
showExplanations	true	Why + concept after each tap
emergencyApps	Phone	Never shielded

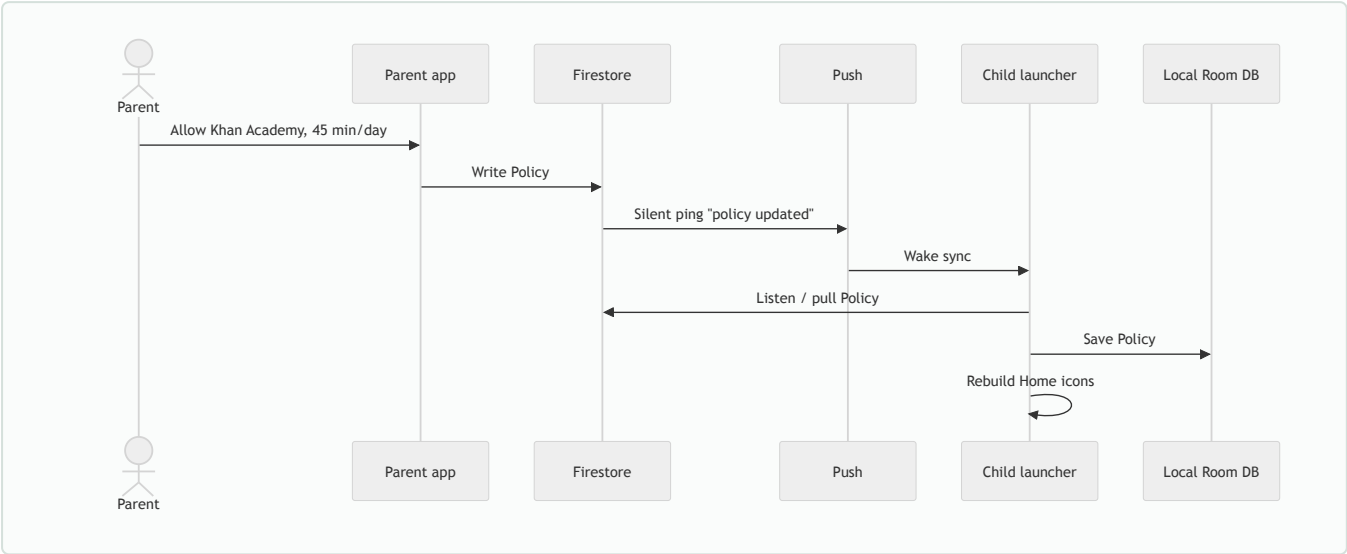
Per allowed app

Setting	Example	Effect
packageOrBundleId	YouTube	Identity on that platform
allowed	true	Icon / launch allowed
blockMinutes	30	Use this long, then quiz
grantOnPassMinutes	30	Next block after a pass (defaults to blockMinutes)
cooldownMinutes	15	On fail, shield all non-emergency apps for this long
isEmergency	false	Skips quiz and cooldown

Runtime state (device, then synced): minutesUsedInBlock , deviceShieldedUntil , blocksGrantedToday .
One device lock, not a per-app shield.

End-to-end data flow

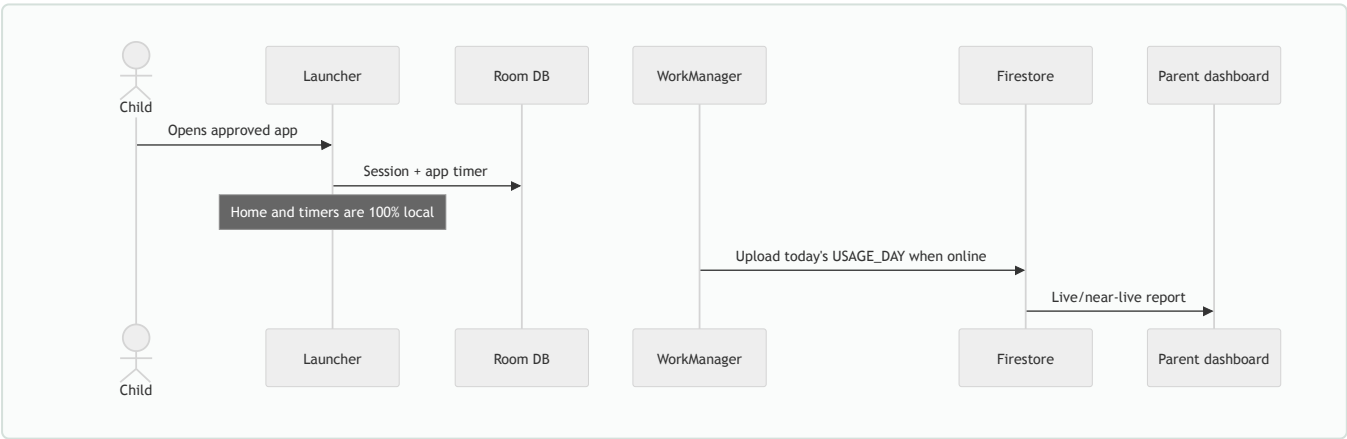
A. Parent changes a rule



Flow diagram 2

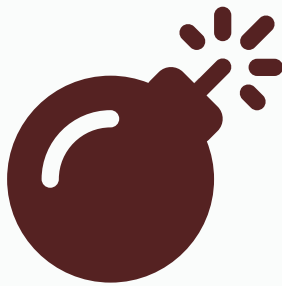
If the child is offline, the **old policy stays in force** until the next successful sync. Home never waits on the network.

B. Child uses the phone (productivity data)



Flow diagram 3

C. Quiz with AI (generation in cloud, playback on device)



Syntax error in text

mermaid version 11.17.2

If AI is down, the **curated question bank** on the device is used (same explainer fields). The launcher never blocks Home on an AI call. Difficulty still moves locally from the last answers.

Quiz trigger modes (when data is written)

Enforcement detail and the YouTube timeline live in [07](#). Summary:

Mode — App block (default, matches the brief)

1. Child uses one approved app until `blockMinutes` (example: YouTube 30).
2. Quiz interrupts that app.
3. Pass → grant `grantOnPassMinutes` on **that app** (example: another 30).
4. Fail → shield **all non-emergency apps immediately**, including the one in use. Only emergency apps stay open. Unlock when `cooldownMinutes` elapse **or** a retry quiz is passed.
5. After unlock, approved apps are available again. The next quiz is at the end of the next block.

Mode — Every session

1. New session (unlock / after idle) or first open of a gated app requires a quiz.
2. Pass → apps allowed for this session.
3. Fail → same device-wide lock: all non-emergency apps until cooldown or a passed retry.

Mode — Daily ceiling (optional, can combine)

1. When today's total minutes hit `dailyCeilingMinutes`, further grants stop.
2. Quiz may still be offered if the parent wants extra time, up to a parent cap.
3. If they fail, the same device-wide lock applies until cooldown or a passed retry. A daily ceiling can still block further grants after unlock.

Sync and conflict rules

Data	Source of truth	Conflict rule
Policy	Parent / Firestore	Latest parent write wins
Usage	Child device	Child minutes cannot be lowered by the parent (parent can only add bonus)
Quiz attempts	Child device	Append-only; never rewrite history
Skill state	Child device	Latest local update wins; cloud copy is for parent + next pack
Pairing	Cloud	Parent revoke immediately invalidates device on next online check; local PIN still works offline

Storage map (where bits live)

Store	What	Why
Android Room / iOS SQLite	Policy, quiz pack, today's timers, skill_state	Instant Home, offline adaptive quiz
Encrypted SharedPreferences / DataStore + Keystore	Pairing secret, PIN verify	Device security
Firestore	Accounts, profiles, policy, usage days, attempts	Parent dashboard + multi-device
Cloud Storage (optional)	Large quiz media (audio for 3–6)	Keep APK small
Crashlytics	Crashes, non-PII	Stability
FCM	Silent policy sync + parent alerts	Fast updates

Parent vs child views of the same data

Data	Parent sees	Child sees
Minutes used	Chart by app, by day	A simple “15 minutes left” bar
Quiz	Score, topic, pass/fail, topic levels	One question; after a miss: why + concept
Allowed apps	Full picker of installed packages	Icons only, no “blocked apps” list
AI packs	Toggle on/off, quality flag	Invisible — just the next quiz
Email / billing	Parent account	Never

Retention (recommended)

Data	Keep
Daily usage summaries	90 days (parent can export)
Raw session ticks	7–14 days then roll up
Quiz attempts	90 days
Account	Until parent deletes family
Child profile	Parent delete = wipe device pairing + cloud child docs

Parent-initiated **Delete family** must remove Firestore child data and instruct the device to wipe local Room (except a “unpaired” flag).

4. 04 — Screens & features

Screens are split by **who holds the phone**.

The Android child experience is a **launcher** (full home). The parent experience is a normal app.

Copy, icons, and tap targets on the child side stay **large and calm**. Ages 3–6 should not need to read paragraphs.

Feature map

Feature	Parent	Child
Account sign-in	Yes	No
Pair device	Shows QR	Scans QR
Set as default launcher (Android)	Reminds if broken	System dialog + PIN to undo
Allowed apps	Edit list + per-app block and cooldown	Sees icons only
App time block	e.g. YouTube 30 min	Remaining time on that app
Fail cooldown	e.g. 15 min	All apps shielded; only Phone works; retry available
Quiz mode	App block, session, or daily ceiling	Quiz interrupts when the rule fires
Adaptive difficulty	On/off (default on)	Next question easier/harder from last answers
Concept explanations	On/off (default on)	After a miss: why + simple concept
AI quiz packs	On/off	No setting
Rewards	Configure	Celebration / sticker
Usage report	Charts	Not shown as a report
Parent PIN	Create / reset	Gate on settings
Crash-free lightweight Home	—	Primary job of the app

Shared / first-run screens

ID	Screen	Purpose
S00	Splash	Fast brand, no network wait
S01	Role select	I am a parent / Set up this child device
S02	Legal & kids privacy	Short COPPA-style notice; parent consent

Parent screens

ID	Screen	What happens
P01	Welcome	Value in one screen: safe home + learning for time
P02	Sign up	Email, Google, Apple
P03	Sign in	Returning parent
P04	Forgot password	Firebase reset
P05	Create family	Family name optional
P06	Add child	Name, age band, language, avatar
P07	Set Parent PIN	4–6 digits, confirm
P08	Pairing	QR + 6-digit code, expiry, refresh
P09	Waiting for device	Spinner until child confirms
P10	First policy setup	Allowed apps, minutes, quiz mode wizard
P11	Dashboard home	Today's minutes, last quiz, device online
P12	Child detail	One child: usage, quizzes, device health
P13	App allowlist	Per app: allow, block minutes, pass grant, cooldown, emergency
P14	Time limits	Daily ceiling, fail cooldown. Fail lock is always all non-emergency apps
P15	Quiz settings	Mode (app block / session / daily), length, pass %, adaptive, explanations, AI
P16	Rewards	Extra minutes, stickers, weekend bonus
P17	Reports	7 / 30 day usage, quiz accuracy, topic levels, weak concepts
P18	Notifications	Daily summary, "time up", "quiz failed 3 times"
P19	Account	Email, PIN reset, add another child, sign out
P20	Delete family	Confirm wipe

Allowlist note: On Android the child device sends the list of launchable apps (name + package + icon hash). The parent picks from that list so we never guess the wrong package.

Child screens (Android launcher)

These replace the system home. Back / Home returns here.

ID	Screen	What happens
C01	Pairing	Camera QR or number pad for 6-digit code
C02	Launcher setup	“Set as Home app” checklist
C03	Permissions	Usage Access (if we use it), notifications optional
C04	Downloading quizzes	First pack; skip continues with built-in bank
C05	Child Home	Approved app grid only. Shielded icons show a countdown. No stock drawer, no widgets
C06	App not allowed	Calm “Ask a parent”. Also used if they reach a shielded app via Recents
C07	Time remaining	Minutes left in this app's block
C07b	Quiz interrupt	Block ended while the app was open — quiz comes to the front
C08	Quiz intro	“3 short questions to keep using YouTube”
C09	Quiz question	One question, large choices, audio optional (3–6)
C09b	Answer + teach	Correct: one-line why. Wrong: why this choice, then the concept
C10	Quiz result pass	Short reward. Returns them to that app for another block
C11	Quiz result fail	Usage stops now. Countdown + Retry quiz. Phone remains
C12	Fail lock	Full-screen lock. Every approved icon disabled. Only emergency apps
C13	Daily ceiling	Only if a daily cap is on and used up
C14	Parent PIN	Numpad overlay
C15	On-device parent menu	After PIN: allowlist, unpair, switch launcher
C16	Bedtime lock (if enabled)	“Sleep time” illustration, PIN override

Child Home (C05) layout — keep it tiny

1. Grid of approved icons only (about 8–20). During a fail lock this grid is replaced by the lock screen — the child cannot pick another app.
2. No search, no stock drawer, no widget wall.
3. Small parent-lock control (opens PIN).

4. Remaining time is about the **app they open**, not a confusing single bar for the whole phone — unless a daily ceiling is also on.

iOS screens (SwiftUI, not a launcher)

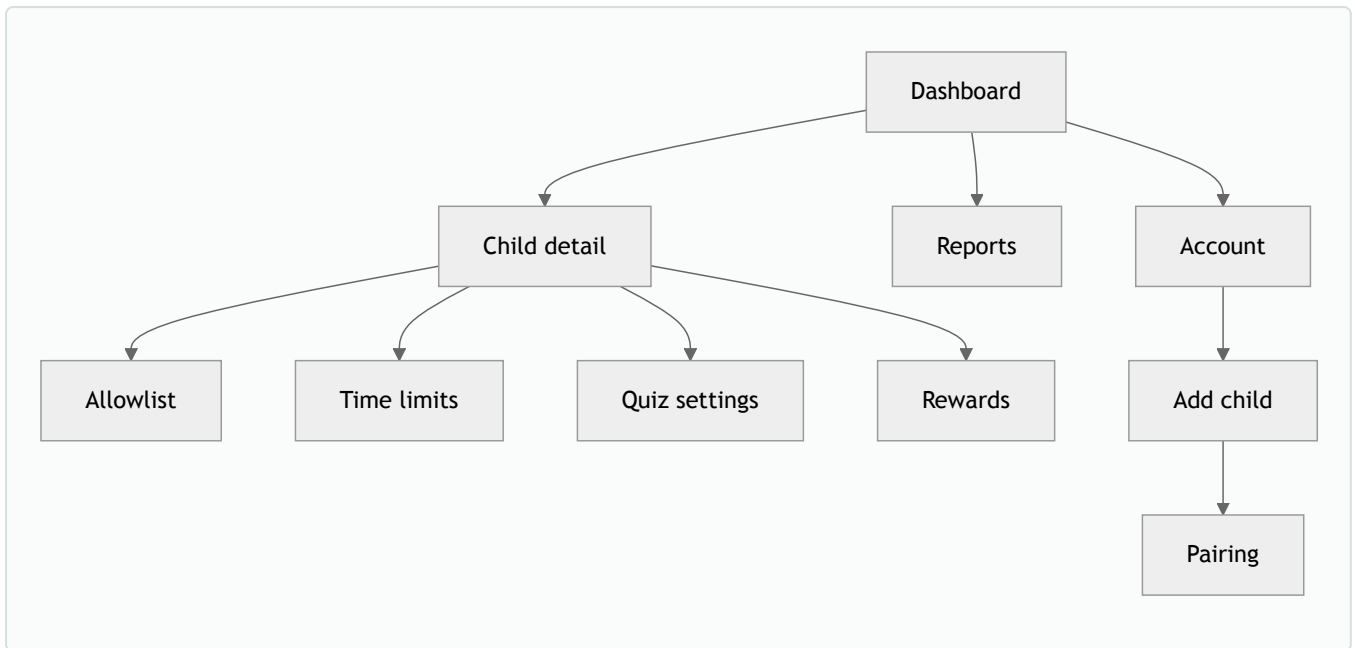
Same pairing, quiz, pass/fail, and cooldown rules (C01, C07b–C12). SpringBoard stays. SwiftUI draws every MeritScreen screen. Family Controls draws the system shield.

ID	Screen	Purpose
I01	Screen Time authorization	FamilyControls authorization (parent-managed child)
I02	Shield	On fail, shield the full allowed set except emergency. Copy: countdown or “Try the quiz again”
I03	Shield action → quiz	Opens the SwiftUI retry. Pass clears every shield and starts the next block
I04	Parent dashboard	Same jobs as P11–P17, SwiftUI on the parent’s iPhone

Quiz UX rules (all ages)

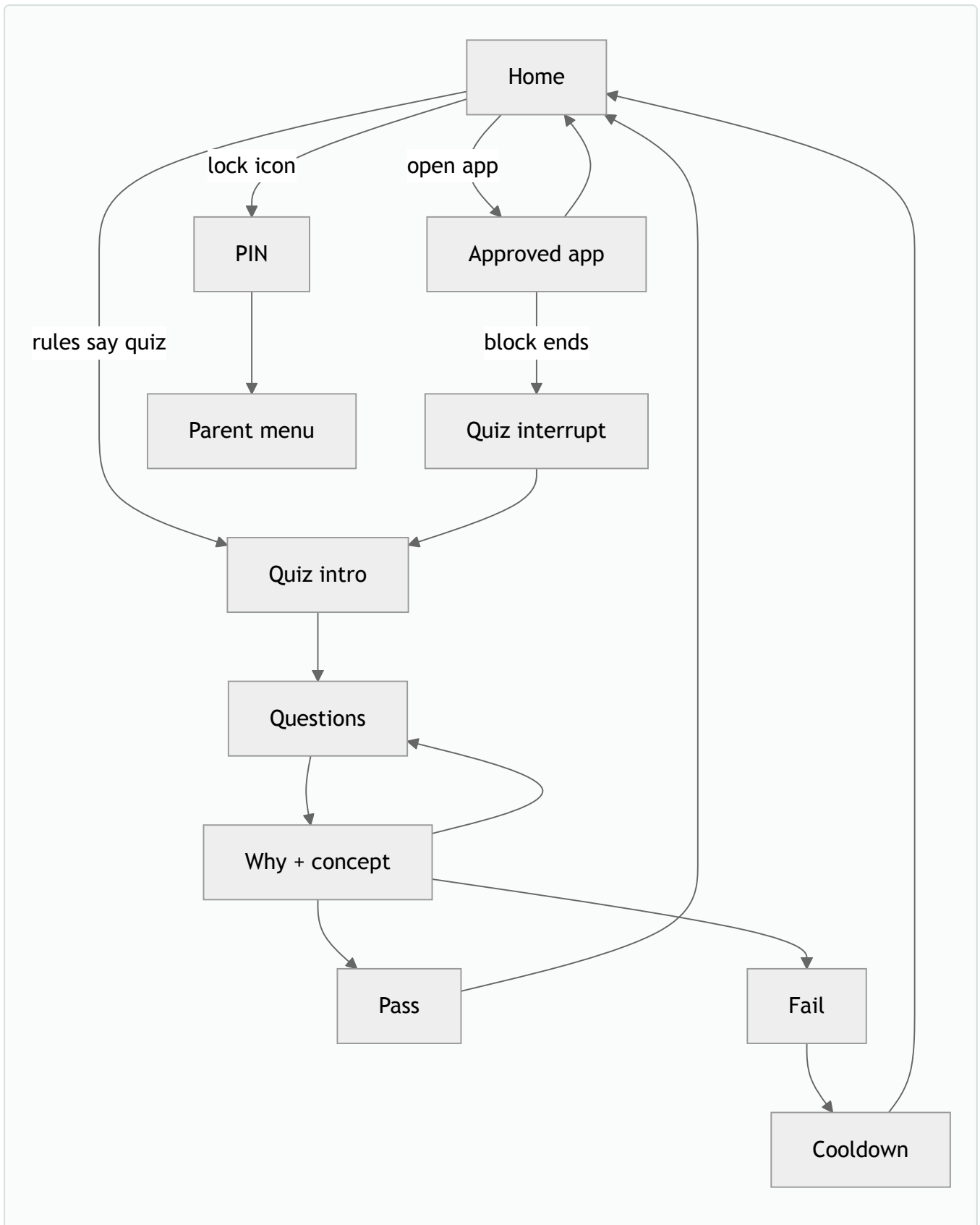
1. **3–5 questions**, never a long test.
2. Ages 3–6: pictures, 2–3 choices, optional spoken prompt **and spoken explainer**.
3. Ages 7–12: text + numbers; still one question per screen.
4. **Adaptive**: first item near saved skill; after a miss, next item is usually an easier check of the **same concept**. After two hits in a row, step up one rung. Never leave the parent’s age band. Details: [Q6](#).
5. **After every tap (C09b)**: if wrong, show why *that* choice fails, then the concept in plain words; if right, one short “yes because”. Copy is already on the device — no spinner.
6. No public leaderboard.
7. Fail is **neutral** (“Let’s rest and try again”), never shame.
8. Questions come from **local pack**. Spinner is only if pack is empty and download is in progress — then fall back to built-in bank immediately.

Navigation (parent)



Flow diagram 1

Navigation (child Android)



Flow diagram 2

Notifications (parent device)

Event	Notification
Child paired	Success
App block ended / quiz due	"Quiz on [child]'s [app]"
Fail cooldown started	"[App] resting for [n] min"
Quiz passed / failed	Optional, batched
Launcher no longer default	"Finish setup" / security
Weekly summary	Minutes + quizzes

Child device: **no marketing notifications**. At most a local notification if the parent enabled "time almost up".

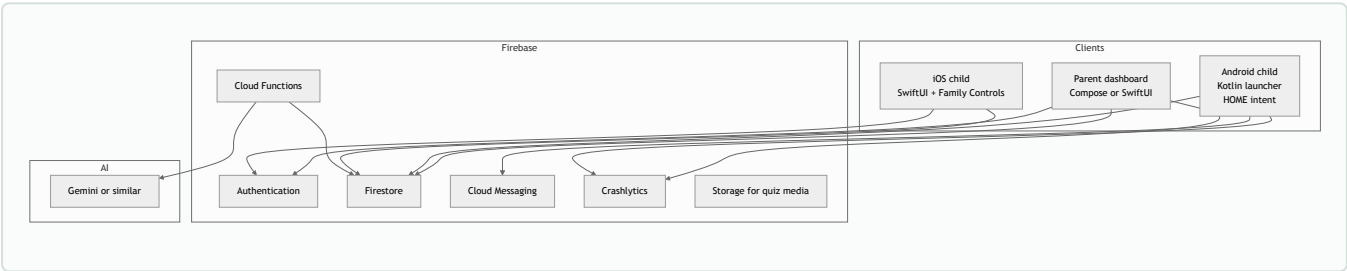
5. 05 — Architecture & performance

MeritScreen is two native clients and one backend.

- **Android child:** Kotlin + Jetpack Compose **launcher**. It replaces the stock Home app.
- **iOS child and parent:** **SwiftUI**. Restrictions use Family Controls. There is no custom Home on iOS.
- **Parent on Android:** the same Kotlin app, a normal (non-launcher) navigation graph.
- **Backend:** Firebase. Policy JSON is shared so an iPhone parent can manage an Android child phone.

AI never runs on the path that paints Home or interrupts an app. Timers and shields are local.

System context



Flow diagram 1

Android child app — why Kotlin native

The process that answers **HOME** must be:

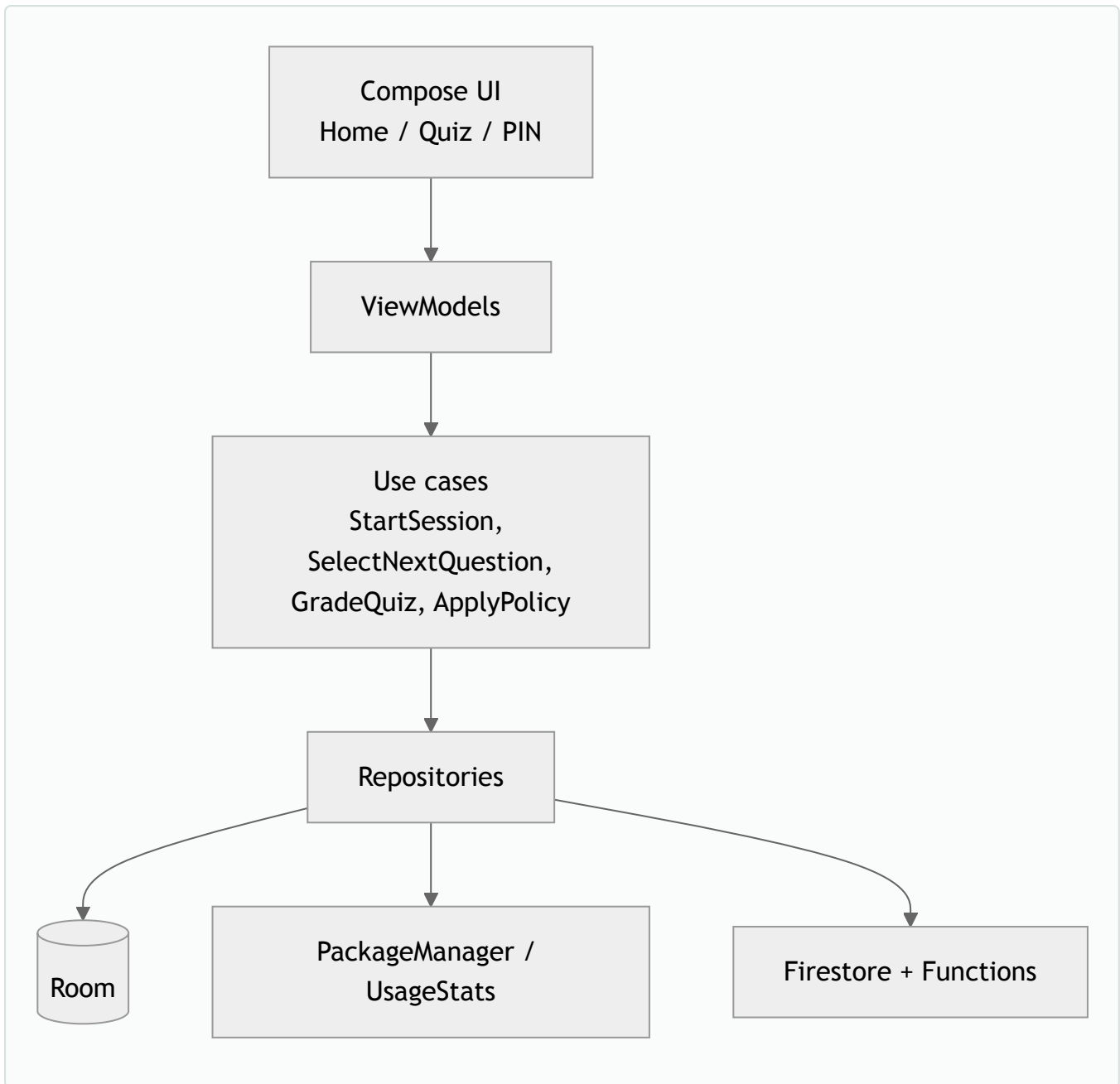
Requirement	Implication
Cold start in tens of ms after first install	Kotlin, Compose, no WebView home
Works with radio off	Room + local policy
Small APK	One launcher module; quiz media on demand
Predictable memory on low-end phones	No on-device LLM
Play Families policy	Minimal permissions, no ads SDK on child

Recommended stack:

Layer	Technology
Language	Kotlin
UI	Jetpack Compose
Home	Activity with HOME / DEFAULT launcher intent filters; ROLE_HOME
App list	PackageManager.queryIntentActivities
Local DB	Room
Prefs	DataStore
Secrets	Android Keystore
Background sync	WorkManager (exponential backoff, idle + connected)
Async	Coroutines + Flow
DI	Hilt
Images	Coil (app icons from PackageManager, not the network)
Analytics (optional)	Firebase Analytics with child-safe events only, or none in v1
Crashes	Crashlytics
Backend	Firebase Auth (parent only), Firestore, Functions, FCM

Parent app on Android can share a Gradle module for models/sync, with a **different UI graph** (no launcher intent).

Architecture layers (Android)



Flow diagram 2

Rule: `HomeViewModel` reads **only Room + PackageManager**.
It does not call Firestore or AI on the main path.

Becoming the launcher (Android)

1. Manifest: `android.intent.action.MAIN` + `android.intent.category.HOME` + `DEFAULT`.
2. After pairing, open the system **Role / default Home** picker.
3. Observe `RoleManager` / default launcher package. If it is no longer us, show C02 again and notify the parent.

4. Do **not** rely on Accessibility or Device Admin to trap the user in v1 — that fights Play policy. PIN + default Home + parent alert is the consumer approach.
5. Always-allowed packages: `dialer`, emergency, maybe settings behind PIN.

Shared session engine (both platforms)

The product rule in [07](#) is implemented as one state machine. Kotlin and Swift each own a copy. They do not share code; they share the **policy JSON** and the same transitions.

States per app: `idle` → `in_block` → `quiz_due` → `granted` or `shielded` → `idle`.

Event	Transition
Launch allowed app	Start block timer if not shielded
Block minutes elapsed	<code>quiz_due</code> . Interrupt the app
Quiz pass	Clear shield. Start a new block of <code>grantOnPassMinutes</code>
Quiz fail	Device lock. Shield every non-emergency app until <code>now + cooldownMinutes</code> or a passed retry
Retry pass (if allowed)	Same as pass. Cooldown ends early
Cooldown elapsed	App may launch. Next quiz is at the end of the new block

Emergency apps never enter `quiz_due` or `shielded`.

Android — interrupting the app

Home replacement alone is not enough. The quiz must appear **when the block ends**, even if YouTube is in front.

1. Child opens the app **from our Home**. We record `packageName` + `blockStartedAt`.
2. A launcher-owned timer (in-process, persisted to Room every 30–60s) counts foreground time. Cross-check with Usage Stats if the permission is granted.
3. At block end, start the quiz Activity (our task). Leaving the quiz without a pass does not restore apps.
4. On fail, disable **every non-emergency icon** and show the lock until cooldown or a passed retry. Recents cannot reopen a blocked app. Phone stays launchable.
5. We do **not** use Accessibility or Device Admin to trap the user. That fights Play policy. Default Home + our Activity + a disabled icon is the consumer approach.
6. If the child (or someone) clears the default Home role, the parent is notified. Until they set us as Home again, we cannot hide the stock drawer — say that plainly in the parent report.

iOS — SwiftUI + Family Controls

Launcher replacement is impossible. Professional equivalent:

Piece	Choice
UI	SwiftUI for parent dashboard, pairing, quiz, explanations, cooldown, account. No UIKit-only screens unless an API requires it
Architecture	SwiftUI + <code>@Observable</code> models, a small session store (SwiftData or SQLite), Firebase iOS SDK
Auth	Firebase Auth. Same custom-token pairing as Android
Pick apps	<code>FamilyControls</code> <code>FamilyActivityPicker</code> (tokens, not raw bundle ids in the parent UI)
Shield	<code>ManagedSettingsStore</code> — on fail, shield the full allowed selection except emergency tokens
Threshold	<code>DeviceActivity</code> schedule / event so the monitor fires when block minutes elapse, including if <code>MeritScreen</code> is not in front
Shield copy	<code>ShieldConfigurationDataSource</code> — app name, quiz or countdown. No shame
Shield button	<code>ShieldAction</code> opens the SwiftUI quiz
Pass	Clear shields. Record the next block locally and in the monitor
Fail	Shield the whole allowed set except Phone until cooldown or a passed retry
Reports	<code>DeviceActivityReport</code> where it helps; our own Firestore usage is what the parent dashboard shows so Android and iOS reports match
Extensions	Device Activity Monitor, Shield Configuration, Shield Action — keep them thin. They write a flag; the SwiftUI app runs the quiz

Limits to design around (do not hide them):

- Cannot replace the iOS Home Screen or hide SpringBoard.
- Family Controls tokens are opaque. The parent picker is the Apple picker, not a custom grid of every bundle id.
- Some timing is extension-driven. Persist shield deadlines so a reboot does not clear a cooldown early.
- Parent and child may be the same Apple family. Authorization is a setup step (I01), not something the child can dismiss to escape.

SwiftUI stays light: no live AI, no large animations on the shield path, quiz text from the local store.

Parent apps

Phone	UI	Notes
Android	Jetpack Compose	Shares models with the launcher module. No <code>HOME</code> intent on the parent graph
iOS	SwiftUI	Same screens as P11–P20. Writes the same Firestore policy

A parent signed in on iPhone can pair and manage an Android child device, and the reverse.

Backend

Job	Where
Sign-in	Firebase Auth
Family, child, policy, usage, attempts	Firestore
Pairing token create/consume	Cloud Functions (callable)
Generate AI quiz pack	Cloud Functions, scheduled or on “pack low”; input is skill snapshot
Safety filter on AI JSON	Cloud Functions: schema, reading level, every choice has whyWrong
Push “policy updated”	FCM data message
Parent email summary	Cloud Function + FCM
Delete family	Callable + recursive delete

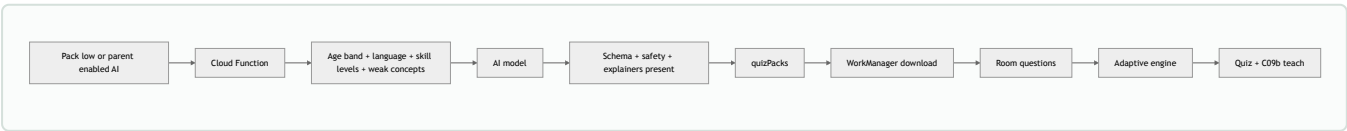
Firestore (logical)

```
users/{uid}
families/{familyId}
  members/{uid}
  children/{childId}
    policy
    appRules/{appId}
    devices/{deviceId}
    usageDays/{yyyy-mm-dd}
    quizAttempts/{attemptId}
    skillState
    quizPacks/{packId}
```

Security rules: only the parent `uid` linked to `familyId` may read/write that family. Child devices use a **restricted custom token** or a dedicated Firebase Auth anonymous/custom account created at pairing — **not** the parent password on the child phone.

Recommended pairing auth: Cloud Function verifies the one-time code and mints a **custom token** scoped to that `deviceId` / `childId`. The child SDK signs in with that token. Rules allow the device to write `usageDays` and `quizAttempts` only for its `childId`, and read only its `policy` and `quizPacks`.

AI quiz pipeline



Flow diagram 3

Performance rule: generation is **asynchronous and off-device**.

The quiz UI and the “why this is wrong” screen read **only Room**.

Fallback order:

1. Current downloaded pack (engine picks by current rung)
2. Built-in bank shipped in APK (same explainer JSON, per age band)
3. Only then: wait for network (with a friendly illustration, never a blocking spinner on Home)

Adaptive engine (Kotlin, local):

- Five rungs (L1–L5) inside the parent’s age band; new topics start at L2
- Two consecutive correct → +1 rung; one wrong → −1 rung and mark `conceptId` weak
- Next item: after a miss, same concept one rung easier; otherwise current rung ± 1
- After each tap show pre-written `whyWrongByChoice` + `conceptExplainer` (or `whyCorrect`)
- Sync `skillState` in WorkManager for the parent report and the next AI pack

Full product rules: [06 — Adaptive quiz & explanations](#).

Do **not** call the model when the child taps a wrong answer. Live tutoring would add latency, cost, and extra child data in the prompt.

Performance budget (child launcher)

Treat this as a product requirement, not a later optimization.

Metric	Target
Home first paint after process start	Under ~300 ms on a mid-range phone (warm), under ~800 ms cold
Scroll of app grid	60 fps, icons from memory / launcher cache
Quiz question / explainer change	Instant; copy from Room, images pre-decoded
APK size (child features)	Keep lean; no Play ML Kit / no ads / no huge Lottie on Home
Memory	Avoid loading all quiz media at once
Network on Home	Zero
Main thread	No Firestore listeners on Home; sync in WorkManager

Practical tactics:

- Baseline profiles + R8
- Compose baseline & avoid heavy blur
- Cache launcher apps list; refresh on `PACKAGE_ADDED` / `REMOVED`
- Count time with `elapsedRealtime` locally; flush to Room every 30–60 s, not every second to disk
- FCM high-priority only for policy; usage upload can be opportunistic

- Image-heavy AI questions stored as compressed WebP in Storage, downloaded in the pack, not at question time

Permissions (Android child) — keep the list short

Permission	Why
Default Home role	Core product
PACKAGE_USAGE_STATS (optional)	Cross-check time if apps leave our launcher
CAMERA	QR pairing only; can skip if parent types the code
INTERNET	Sync, not Home
POST_NOTIFICATIONS	Optional parent-driven local alerts
QUERY_ALL_PACKAGES	Only if Play allows for launcher; otherwise query launcher intents

Do not request location, contacts, SMS, or microphone.

Store compliance (from the original brief)

- **Google Play Families / Designed for Families** if the child APK is aimed at under-13
- **COPPA / GDPR-K**: parental consent at P02/S02, minimal data, delete family
- **App Store** kids category + Screen Time entitlements for iOS
- No third-party ads on the child experience
- Privacy policy must list exactly the tables in [03 — Data model](#)

Build sequence (recommended)

Both clients are in scope. Build the shared rule first, then each shell.

1. Shared policy JSON + session states (app block, pass grant, fail shield, cooldown, retry).
2. Android: pairing, Kotlin Home, allowlist, PIN, **quiz interrupt when the block ends**.
3. Local quiz bank, adaptive engine, explainers (works offline on both later).
4. Firebase parent dashboard (Compose) writing per-app rules.
5. iOS: SwiftUI parent + child, Family Controls picker, DeviceActivity threshold, ManagedSettings shield, same quiz JSON.
6. AI packs that follow `skillState`.
7. Play Families + App Store kids submission.

Do not ship a per-app-only fail. A failed quiz must stop all further usage except emergency apps until cooldown or a passed retry.

6. 06 — AI adaptive difficulty and concept explanations

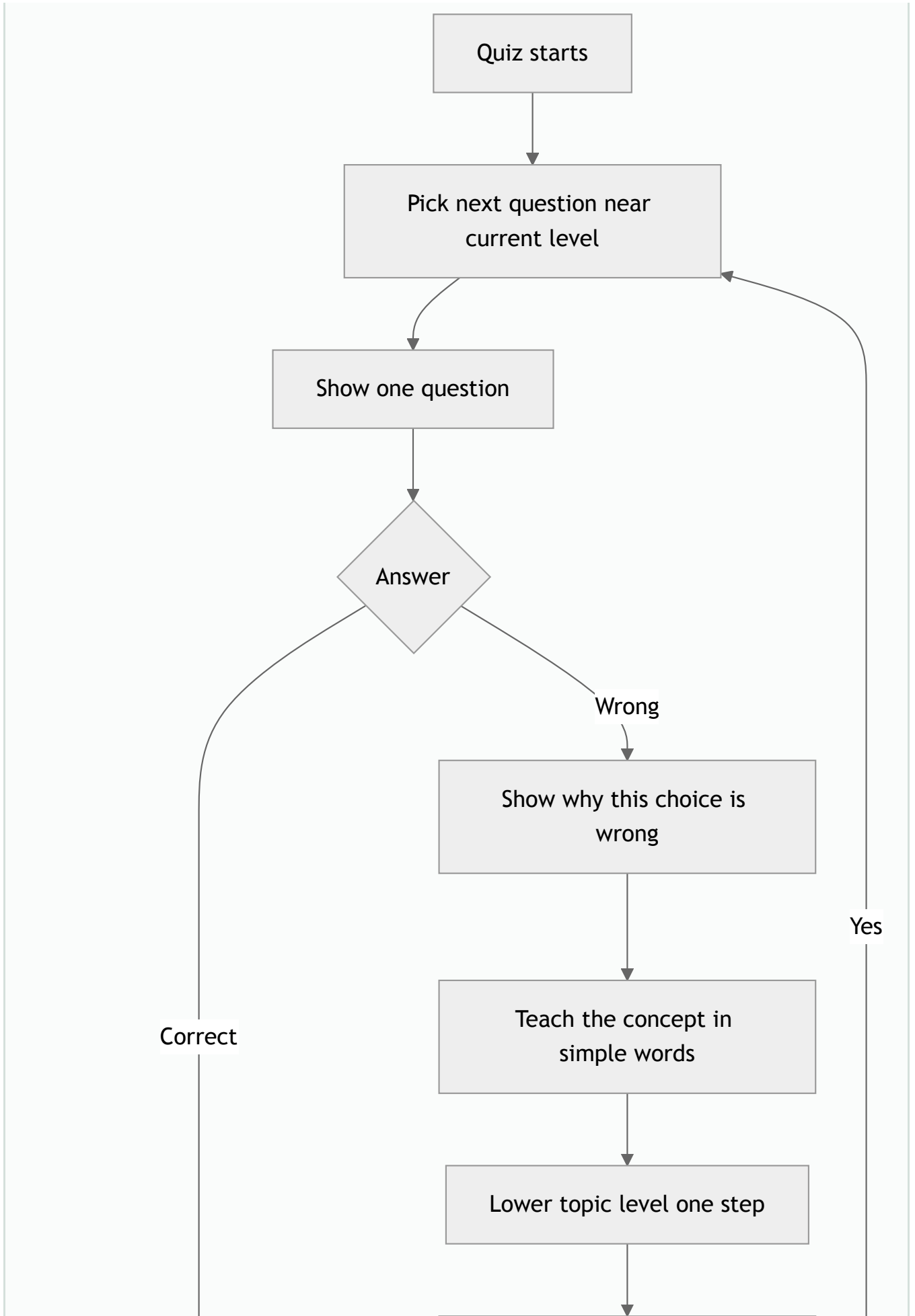
Every quiz **learns from the last answers**.

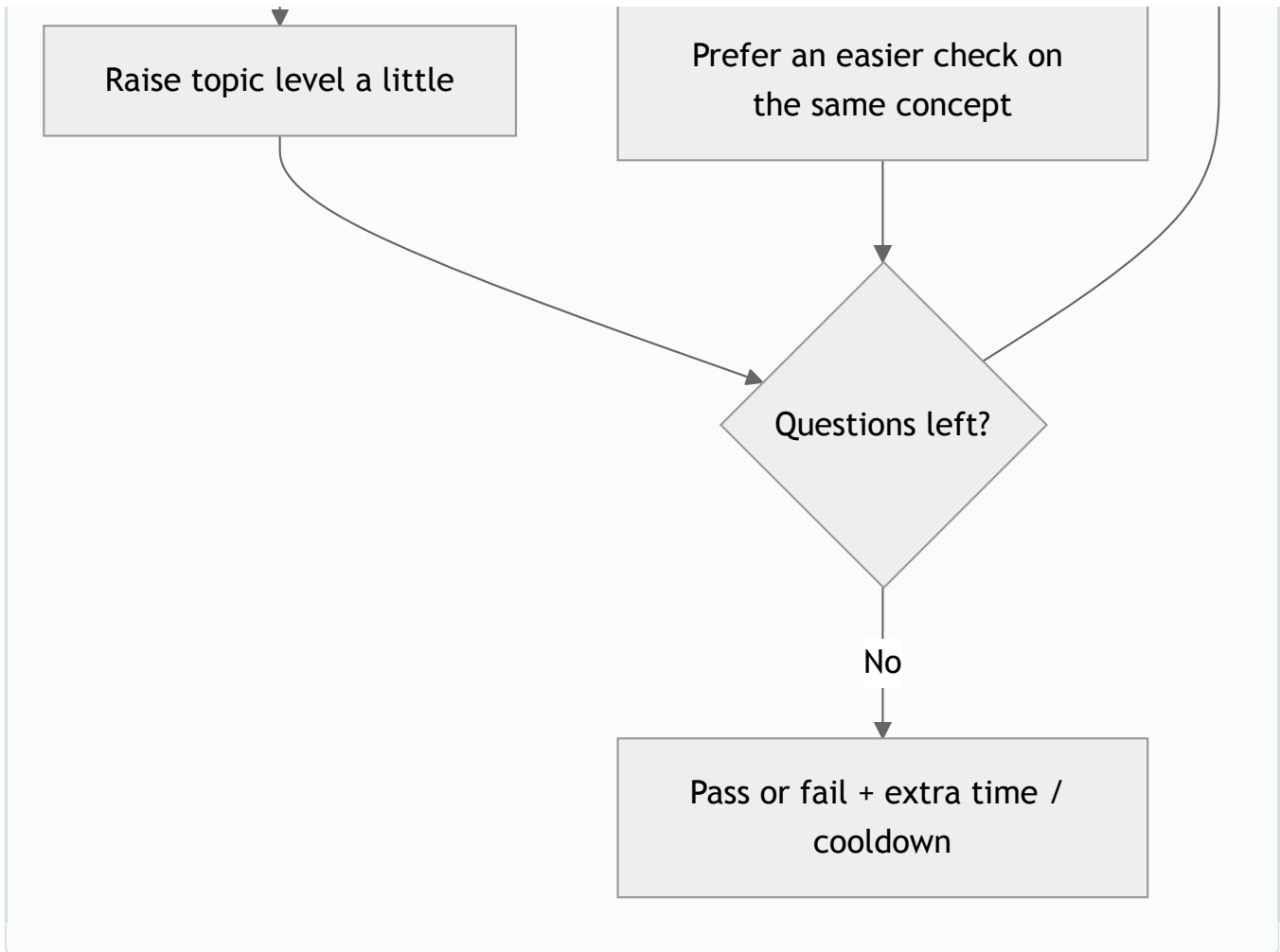
If the child is wrong, the app **does not only say “incorrect”** — it explains *why*, then teaches the **concept** in short, age-fit language. The next question gets easier or harder from that result.

This still respects the launcher rule: **no AI call while the child is looking at Home or a question**.

Adaptation on the phone is a small local engine. AI only **pre-writes** the next pack (questions + explanations) in the cloud.

What the child experiences





Flow diagram 1

1. First question in a session uses the child's **saved skill** (or a mid level for that age band if they are new).
2. After **correct**: a short confirm, then a slightly harder or same-level question.
3. After **wrong**:
 - Calm line: this one is not right.
 - **Why this answer is wrong** (tied to the choice they tapped).
 - **What the idea is** (the concept, not a lecture).
 - Continue. The next question is usually an **easier check of the same concept**.
4. End of quiz still decides extra screen time. Teaching happens **during** the quiz, not only on the fail screen.

Ages 3–6 get pictures and very short sentences (optional spoken explanation from the pack). Ages 7–12 can read a few lines. Never shame, never a wall of text.

Two layers (so the phone stays fast)

Layer	When it runs	What it does
On-device engine (Kotlin)	Every answer, instantly	Updates skill, picks the next local question, shows the pre-written explanation
Cloud AI pack	Background, when packs run low	Writes new questions and explanations aimed at current level and weak concepts

The child never waits on a model to hear “why this is wrong”. That text was generated **with the question**, stored in Room, and shown offline.

Skill model (simple, stable)

Age band is the **range**, not a fixed difficulty. Inside each band we use five rungs:

Rung	Meaning inside that age band
L1	Easiest items for this age
L2	Default start for a new child
L3	On track
L4	Stretch
L5	Hardest we allow for this age (never jump to the next age band automatically)

Each **topic** (and each **concept** under it) has its own rung, plus a short streak.

Signal	Engine does
Two correct in a row at this rung	Topic +1 (max L5)
One wrong	Topic –1 (min L1); that concept is marked weak
Correct but slow (parent-optional)	Keep rung; do not punish
New child / new topic	Start at L2 for the age band

Never jump more than **one rung per question**.

Never auto-promote a 6-year-old into the 7–9 bank. The parent can change age band; the engine does not.

Session difficulty = typical (median) rung of topics used in the last few quizzes. The first question of a new quiz sits near that value.

How the next question is chosen (local)

From the unused questions already on the device:

1. If the last answer was **wrong**, prefer same `conceptId` , **one rung easier**, not shown in this quiz.
2. Else prefer a question whose difficulty is **current topic rung ± 1** .
3. Mix topics so a quiz is not five of the same item.
4. Skip question ids already used recently (repeat window, e.g. 20 questions).
5. If the pack has no match, take the closest remaining item, then the built-in bank.

This is ordinary filtering in Kotlin over a small list. No on-device neural net.

Wrong-answer explanation (product copy rules)

Show **immediately after the tap**, on the same quiz flow (new screen **C09b**), before the next question.

Block	Purpose	Length
Result	“Not this one” / “That’s it”	1 line
Why this choice	Why <i>their</i> tap does not fit	1–2 sentences
Concept	The idea in plain words	1–3 sentences (3–6: one sentence + picture)
Continue	Next question	One button

Correct answers: a short “Yes — because ...” (one sentence from `whyCorrect`). Do not skip teaching on a lucky guess.

Tone:

- Professional and kind. No “wrong, silly”, no sarcasm, no scores shouted at ages 3–6.
- Same language as the quiz (`language` on the profile).
- No extra facts that were not needed to understand this item.

Example (age 7–9, math):

- Question: $8 + 5$
- Child taps 12
- Why: $8 + 5$ is not 12. $8 + 4$ is 12, so we still need one more.
- Concept: Adding means joining two groups. Count on from 8: 9, 10, 11, 12, 13.

What AI generates (each question in a pack)

AI does **not** receive the child’s name, apps, or raw usage. It receives a **skill snapshot**:

```
{
  "ageBand": "7-9",
  "language": "en",
  "globalLevel": 3,
  "topics": [
    { "id": "addition", "level": 2, "weakConcepts": ["add-within-20"] },
    { "id": "spelling", "level": 4, "weakConcepts": [] }
  ],
  "avoidQuestionIds": ["q_1021", "q_0888"],
  "needCount": 24
}
```

Each returned item **must** include explanations (pack is rejected if they are missing):

```
{
  "id": "q_2044",
  "topic": "addition",
  "conceptId": "add-within-20",
  "conceptTitle": "Adding by counting on",
  "difficulty": 2,
  "prompt": "What is 8 + 5?",
  "choices": [
    { "id": "a", "text": "12", "correct": false },
    { "id": "b", "text": "13", "correct": true },
    { "id": "c", "text": "14", "correct": false }
  ],
  "whyCorrect": "8 + 5 is 13. Count on five from eight.",
  "whyWrongByChoice": {
    "a": "12 is 8 + 4. We need one more than that.",
    "c": "14 is 8 + 6. That is one too many."
  },
  "conceptExplainer": "Adding joins two groups. Start at the bigger number and count up."
}
```

Cloud Function then:

1. Schema check (every wrong choice has a why).
2. Safety / age filter (no scare, no personal data, reading level).
3. Dedup against `avoidQuestionIds`.
4. Write `quizPacks/{packId}`.

Built-in APK bank uses the **same JSON shape**, written by us, so offline explanations always work even if AI is off.

Data stored for adaptation

On device (Room) — source of truth for the next tap

Store	Fields
skill_state	topicId , conceptId , level 1–5, streakCorrect , streakWrong , seenCount , correctCount , updatedAt
recent_question_ids	Last N ids, to avoid repeats
quiz_item	Full question + explainers

Uploaded (Firestore) — parent report + next AI pack

Store	Fields
skillState (one doc per child)	Per-topic levels, weak concepts, globalLevel
quizAttempts	Per question: chosen choice, correct?, levelBefore , levelAfter , time-to-answer; plus session pass/fail

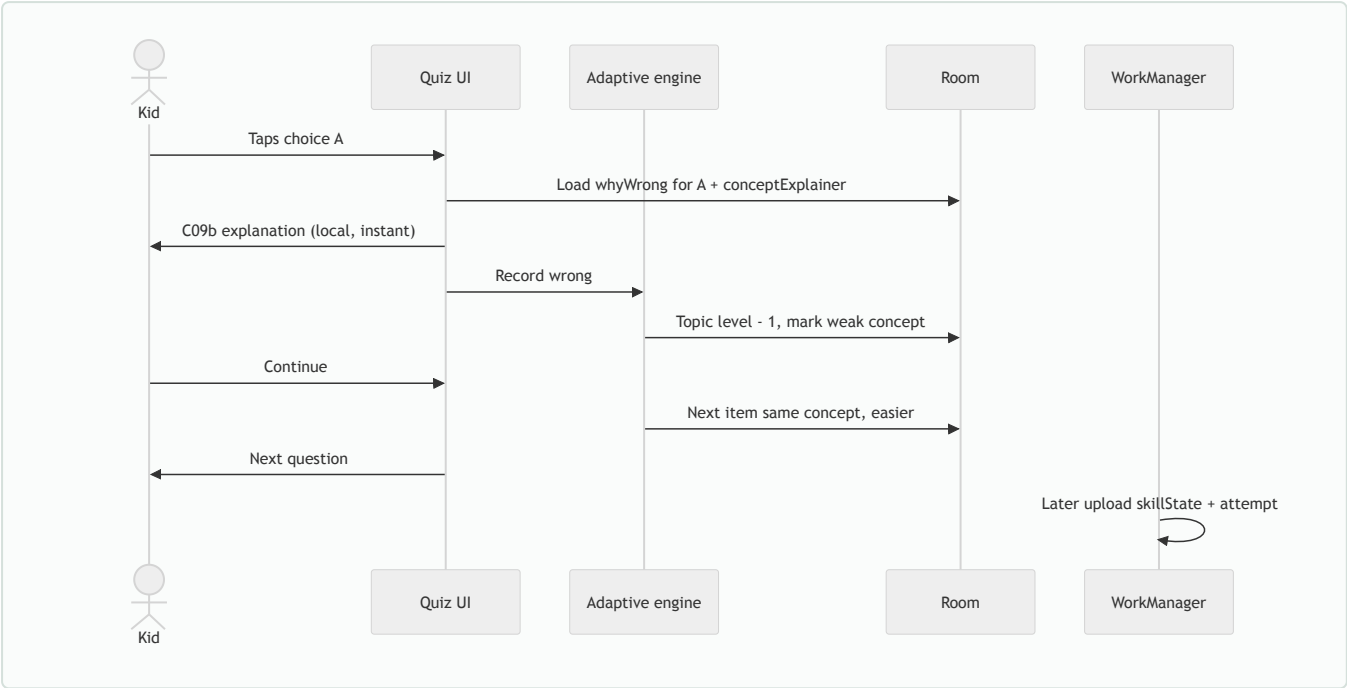
Parent sees **topics getting easier/harder** and **concepts to practice**, not every word of the explainer unless they open a detail row.

Parent controls (P15)

Setting	Default	Effect
Adaptive difficulty	On	Engine moves rungs; next AI pack uses skill snapshot
Show explanations	On	C09b after each answer
AI new packs	On	Cloud fills packs; bank always remains fallback
Age band	Set at profile	Hard ceiling / floor for rungs

If adaptive is **off**, every question stays at L2 for the age band. Explanations can stay on.

Sequence (one wrong answer, then a better-fit question)



Flow diagram 2

Performance and privacy

- Grading and explainers are **Room reads**. Target: explanation screen in one frame, no spinner.
- Do not call Cloud Functions from C09 / C09b.
- Prompt payload is skill numbers and topic ids only.
- Ages 3–6: cap explainer length in generation (e.g. 12 words) so the UI stays large and calm.
- Optional TTS audio is a file in the pack, not a live speech API on the phone.

Failure behaviour

Case	What happens
AI pack delayed	Use bank + last skill; still adapt rungs locally
Missing whyWrong on an item	Show conceptExplainer only; do not block the quiz
Skill doc missing	Start L2; create skill rows as questions are answered
Parent turns adaptive off	Keep showing explainers if that toggle stays on

Build note

Ship the **local engine + bank explainers** with the first quiz.

Turn on **AI packs that follow** `skillState` when Firebase is live.

Do not wait for a live “chat tutor” — that would make the launcher slow and would send too much child data to the model.

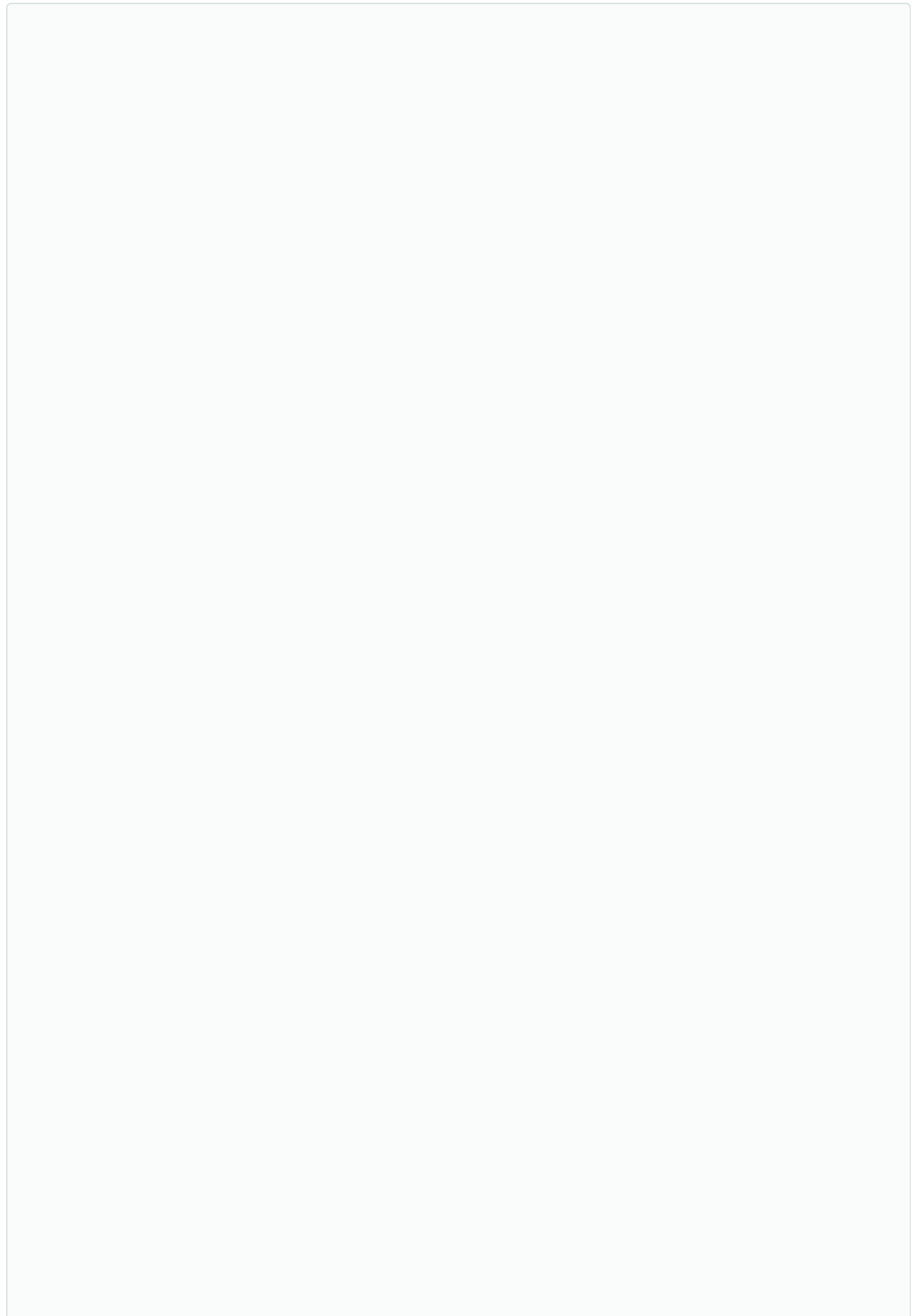
7. 07 — App blocks, quiz gate, and fail lock

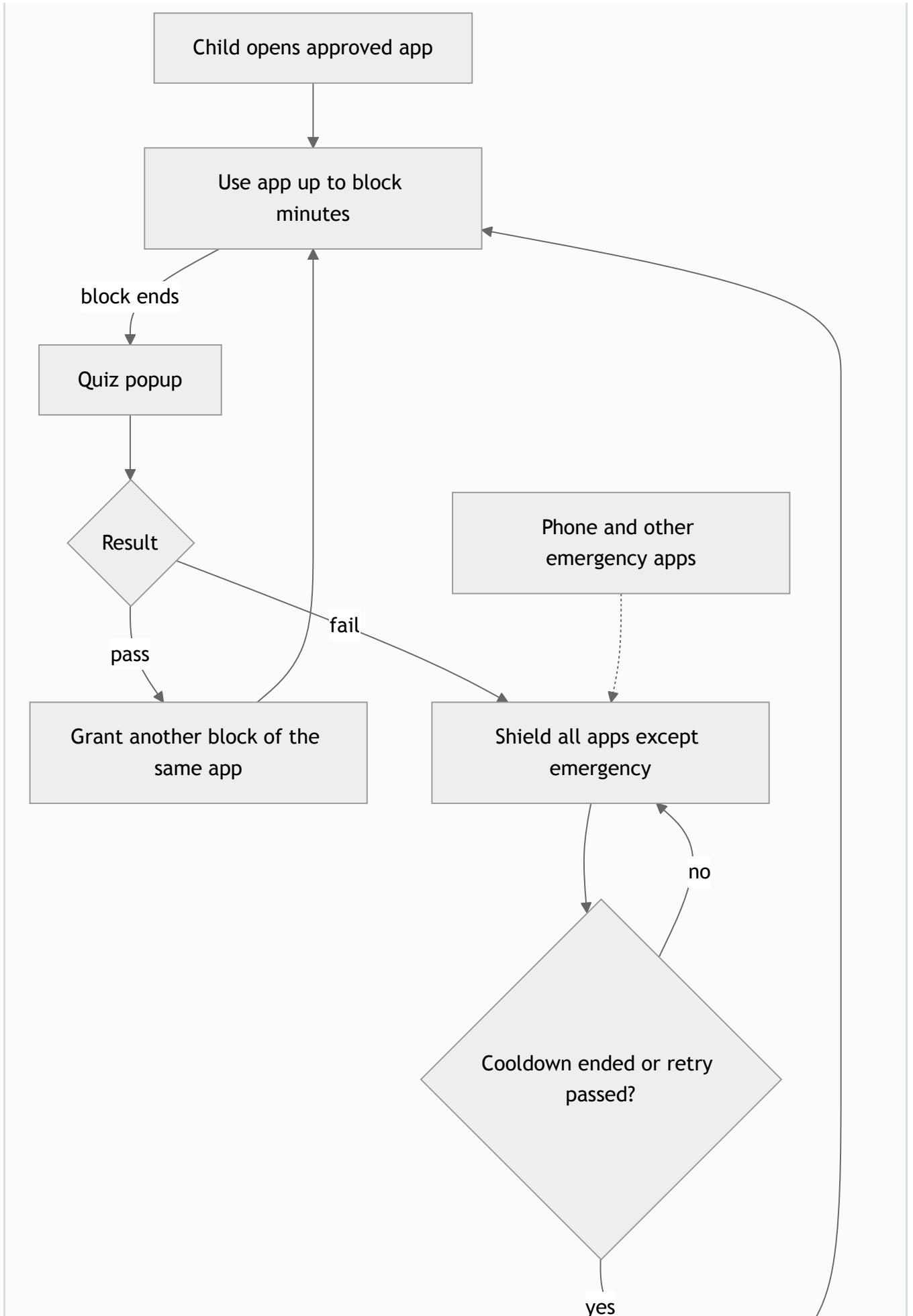
This is the **enforcement rule** the product must match. It is the same on Android and iOS. Only the shield mechanism differs.

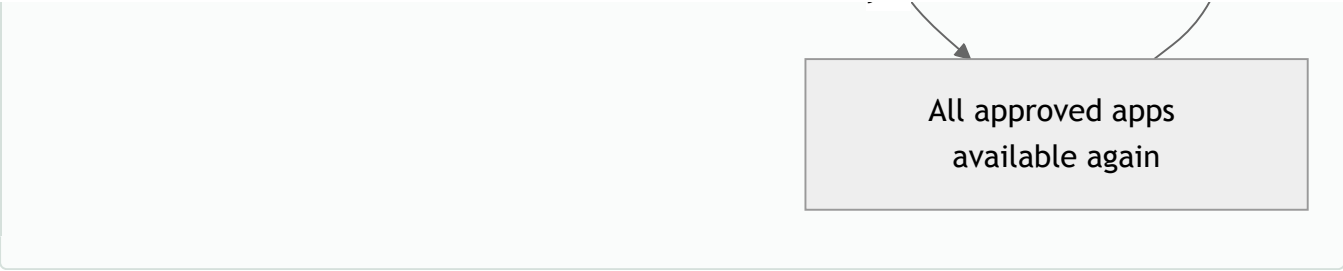
Canonical example (must match)

Parent sets **YouTube = 30 minutes per block, cooldown on fail = 15 minutes**.

1. Child opens YouTube from the approved home (Android) or from the allowed set (iOS).
2. They can watch for **30 minutes**.
3. At 30 minutes a **quiz pops up**. YouTube is paused / shielded until the quiz ends.
4. **Pass** → they may continue YouTube for **another 30 minutes**. Stickers / extra reward if the parent enabled them.
5. **Fail** → **every non-emergency app is shielded immediately**, including YouTube and any other approved app. Only Phone (and other parent-marked emergency apps) stay open. Further usage stops until the rules allow access again.
6. The shield ends only when **either** the parent's cooldown finishes (example: 15 minutes) **or** the child **passes a retry quiz**. A failed retry does not unlock anything. After unlock, approved apps are available again and the next quiz is at the end of the next block.







All approved apps
available again

Flow diagram 1

What is limited

Limit	Meaning	Default
App block	Minutes of one app before a quiz	30 per approved app (parent can override per app)
Pass grant	Minutes added to that app after a pass	Same as the app block (example: +30), unless parent sets a different grant
Fail cooldown	How long all non-emergency apps stay shielded after a fail	15 minutes (parent-set)
Daily ceiling	Optional cap on all apps combined	Off by default; parent can turn on (e.g. 120 min/day)
Always allowed	Never blocked by quiz or cooldown	Phone / emergency. Parent can add one or two more

A fail is a **device-wide stop**. It is not “only YouTube rests while Khan Academy keeps working.”

Quiz trigger modes

The parent picks one primary mode. App-block is the mode that matches the YouTube example.

Mode	When the quiz appears	Pass	Fail
App block (recommended)	When that app’s block minutes are used	Another block of that app	Shield all non-emergency apps until cooldown or a passed retry
Every session	Before a new session (unlock / after idle), or before opening a gated app	Session / open allowed	Same fail lock: all non-emergency apps
Daily ceiling	When today’s total minutes hit the cap	Extra minutes (parent cap)	Same fail lock until cooldown or retry, and no further grants past the ceiling

App block and daily ceiling **can both be on**. The tighter rule wins. Example: YouTube block is 30, daily ceiling is 90. After three passed YouTube blocks, the daily ceiling stops further grants even if they would pass another quiz.

Fail lock (required)

A failed quiz **immediately stops further usage**. This is the v1 rule on Android and iOS. There is no per-app-only fail mode.

What gets shielded

- The app they were using.
- Every other approved app.
- Recents, the stock drawer (Android has none, because MeritScreen is Home), and any second way back into a blocked app.

What stays open

- Emergency apps only. Default: **Phone**. Parent may mark one or two more (for example a parent-contact shortcut). Nothing else.

The child sees one calm full-screen lock: countdown, and a **Retry quiz** button. Copy is neutral (“Let’s rest, or try the quiz again”). No shame.

Unlock paths (either one ends the lock)

Path	What happens
Cooldown ends	Parent-set timer hits zero. All approved apps unlock. The next quiz is at the end of the next block, not immediately.
Retry passed	Child takes the quiz again during cooldown. Pass clears the shield now and grants the next block on the app they were using. Fail keeps the shield. Cooldown restarts from the same length so a child cannot hammer retries.

There is no third path. Closing the quiz, pressing Home, or force-stopping an app does not unlock usage.

Where the quiz appears

The quiz must interrupt the app, not wait until the child happens to press Home.

Platform	How the popup is shown
Android	Launcher-owned timer. When the block ends, MeritScreen brings the quiz Activity to the front. On fail, every non-emergency icon is disabled and Home shows the cooldown lock. Recents cannot reopen a shielded app.
iOS	DeviceActivity threshold fires. On fail, ManagedSettings shields the full allowed set except emergency apps. Shield action opens the SwiftUI retry quiz. Pass or cooldown clears the whole shield.

Offline: timers, shield state, and quiz content are local. Cloud sync of the result can wait.

Worked timeline

Clock	What the child can do
0:00	Opens YouTube. Block = 30.
0:30	Quiz. Answers correctly.
0:30–1:00	YouTube again (second block).
1:00	Quiz. Answers incorrectly.
1:00–1:15	All apps shielded. Only Phone works. Child may retry the quiz.
1:15	Cooldown ends (or retry already passed). Approved apps unlock. New blocks can start.
1:45	Quiz again. Cycle continues until a daily ceiling (if set) or bedtime.

What the parent configures (per child, per app)

Field	Example
Allowed	Yes / no
Block minutes	30
Minutes granted on pass	30 (defaults to block minutes)
Cooldown on fail	15 (applies to all non-emergency apps)
Emergency	No (Phone = yes)

Global:

Field	Example
Quiz mode	App block
Retry during cooldown	On (required unlock path)
Fail lock scope	All non-emergency apps (not optional)
Daily ceiling	Off, or 120
Questions per quiz / pass %	3–5 / 70
Rewards	Stickers, weekend bonus

Reports the parent sees

- Minutes today, **by app**
- How many blocks were granted vs failed

- Current lock: cooldown remaining, or retry in progress
- Quiz score, weak topics (from the adaptive engine)

The child does not see this report. During a fail they see one lock screen: countdown, Retry, and Phone. They do not get a grid of other apps.